

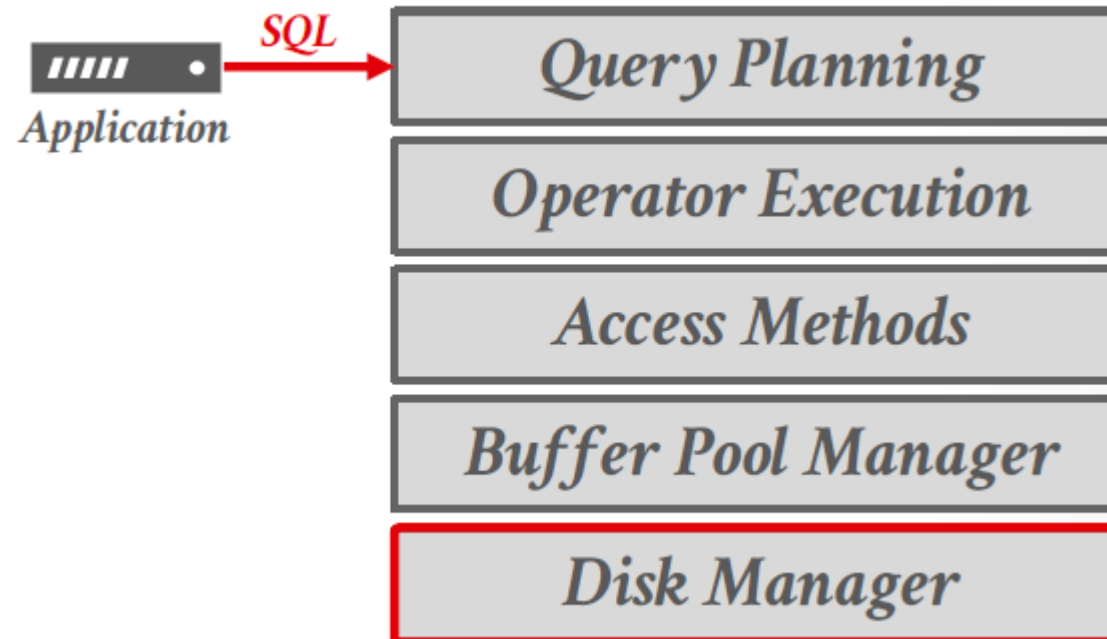
Curso _____
Estructuras de datos

**UNIVERSIDAD
DE ANTIOQUIA**

Clase 6 – Indexación

Contextualización

- El **DBMS** se organiza en capas funcionales, desde el almacenamiento físico (**Disk Manager**) hasta la planificación de consultas (**Query Planning**)
- El **Disk Manager** — capa ya estudiada previamente — es responsable de gestionar el **almacenamiento físico** de los datos en disco, administrando la lectura y escritura de bloques. Se relaciona directamente con el **Buffer Pool Manager**, quien actúa como intermediario trayendo esos bloques del disco a la memoria principal, manteniéndolos en caché para reducir los costosos accesos a disco y entregarlos eficientemente a las capas superiores.



Jerarquia de almacenamiento

Archivo (File DB)

↳ **Pagina (Page)**

↳ **Registro (Record)**

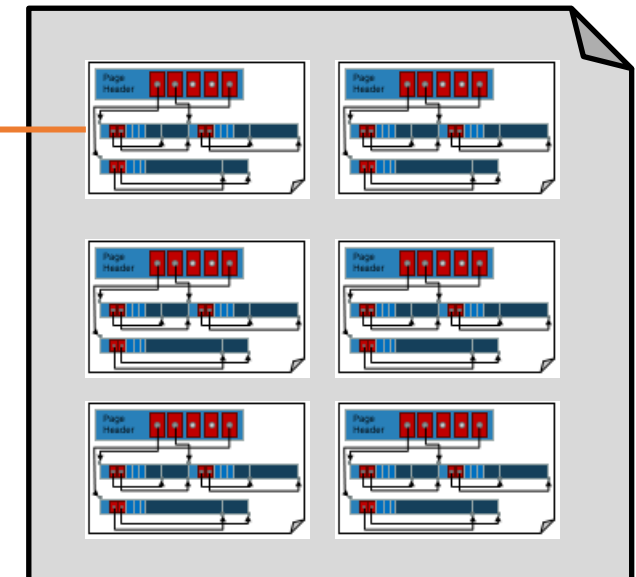
↳ **Campo (Field)**

Relation (table)

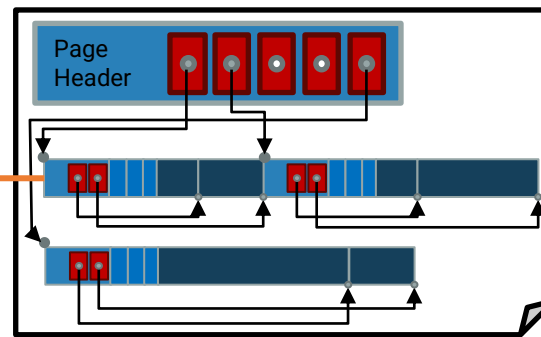
ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400



File



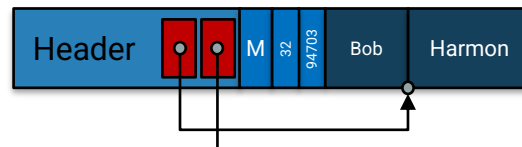
Slotted Page



Record

Bob	Harmon	M	32	94703
varchar	varchar	char	int	int

Byte Representation of Record



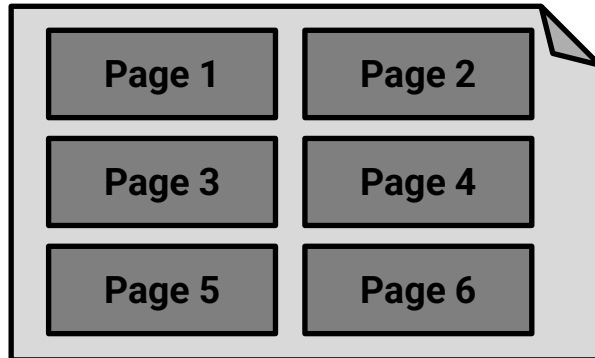
Organización de los archivos

Existen muchas alternativas, cada una ideal para ciertas situaciones, y no tan adecuadas en otras.

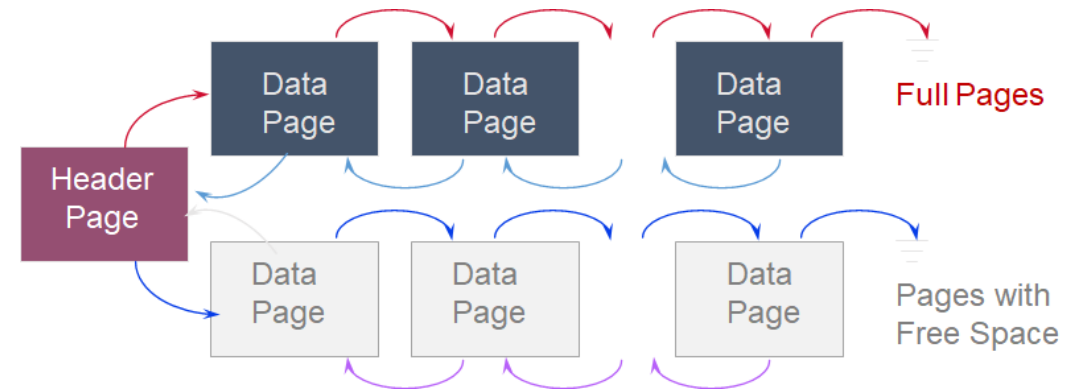
- **Archivos tipo heap (Heap):** Adecuados cuando el acceso típico es un escaneo completo del archivo que recupera todos los registros.
- **Archivos ordenados (Sorted Files):** Son mejores si los registros deben recuperarse en algún orden, o si solo se necesita un rango de registros.
- **Índices (Indexes):** Estructuras de datos para organizar registros mediante árboles (trees) o hashing (hashing).
 - Al igual que los archivos ordenados, aceleran las búsquedas para un subconjunto de registros, basándose en valores de ciertos campos (**search key**).
 - Las actualizaciones son mucho más rápidas que en archivos ordenados.

Archivos tipo heap

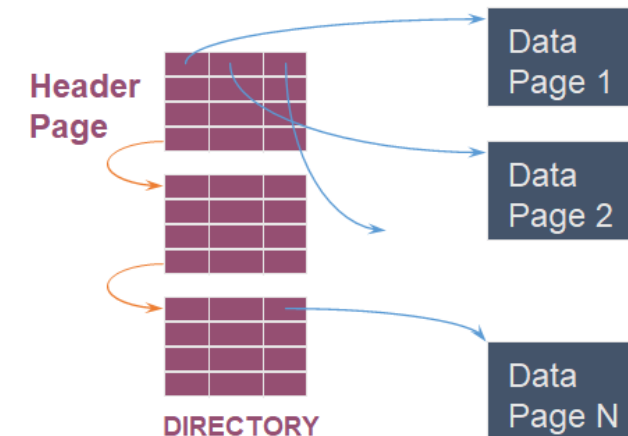
File



Lista
enlazada



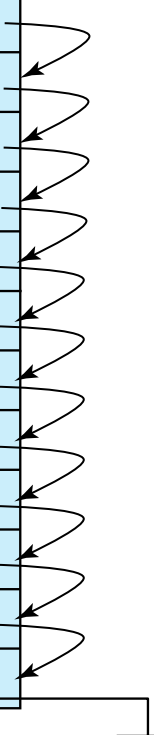
Lista de
directorios



Archivos ordenados (sorted files)

- Adecuado para aplicaciones que requieren procesamiento secuencial de todo el archivo.
- Los registros en el archivo están ordenados según una clave de búsqueda (**search-key**).

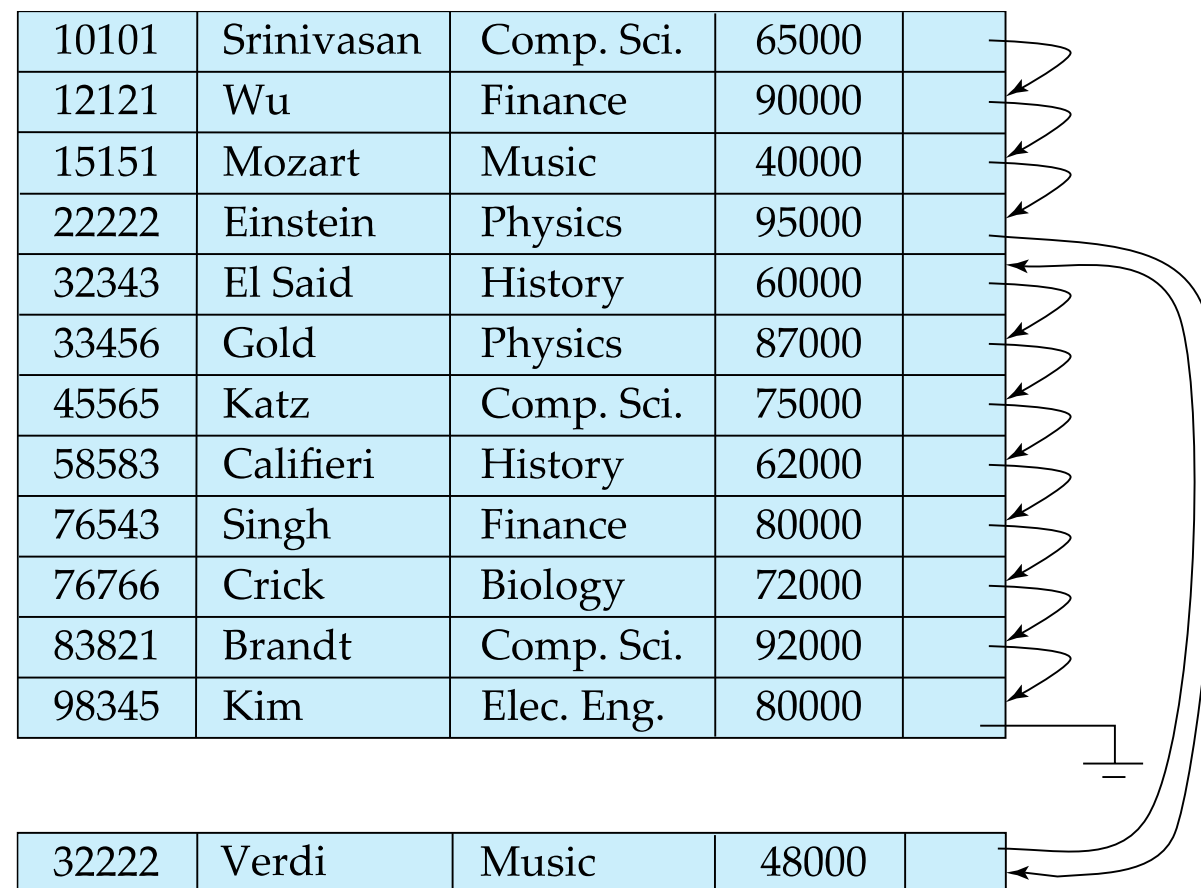
10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Archivos ordenados (sorted files)

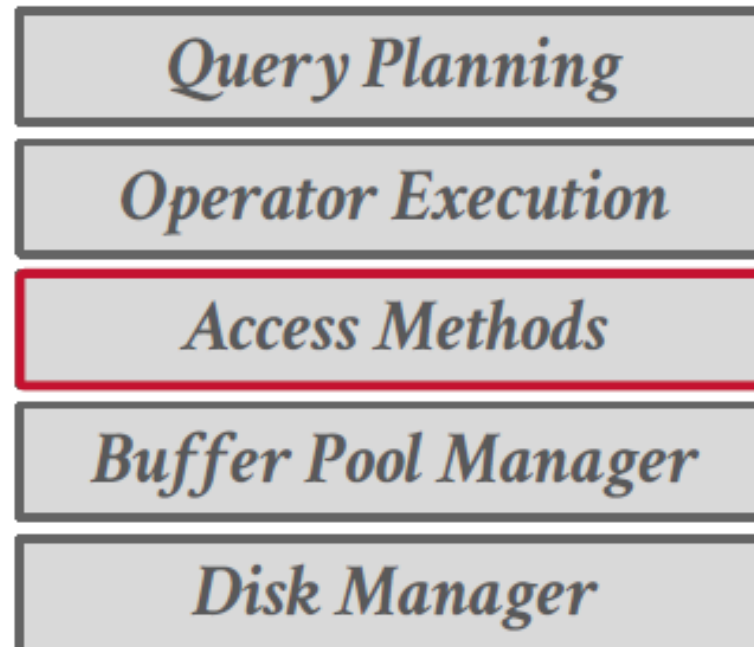
Algunas Operaciones

- **Eliminación (Deletion):** utiliza cadenas de punteros (pointer chains).
- **Inserción (Insertion):** Localiza la posición donde debe insertarse el registro:
 - Si hay espacio libre (free space), se inserta allí
 - Si no hay espacio libre, se inserta el registro en un bloque de desbordamiento (**overflow block**)
 - En cualquier caso, la cadena de punteros debe ser actualizada.
- Es necesario reorganizar el archivo de vez en cuando para restaurar el orden secuencial.



Contextualización

- El **DBMS** se organiza en capas funcionales, desde el almacenamiento físico (**Disk Manager**) hasta la planificación de consultas (**Query Planning**)
- La capa de **Access Methods** es la responsable de leer y escribir datos eficientemente desde las páginas (pages)
- La indexación (**indexing**) es el mecanismo fundamental de esta capa: permite acceder a los datos sin recorrer el archivo completo

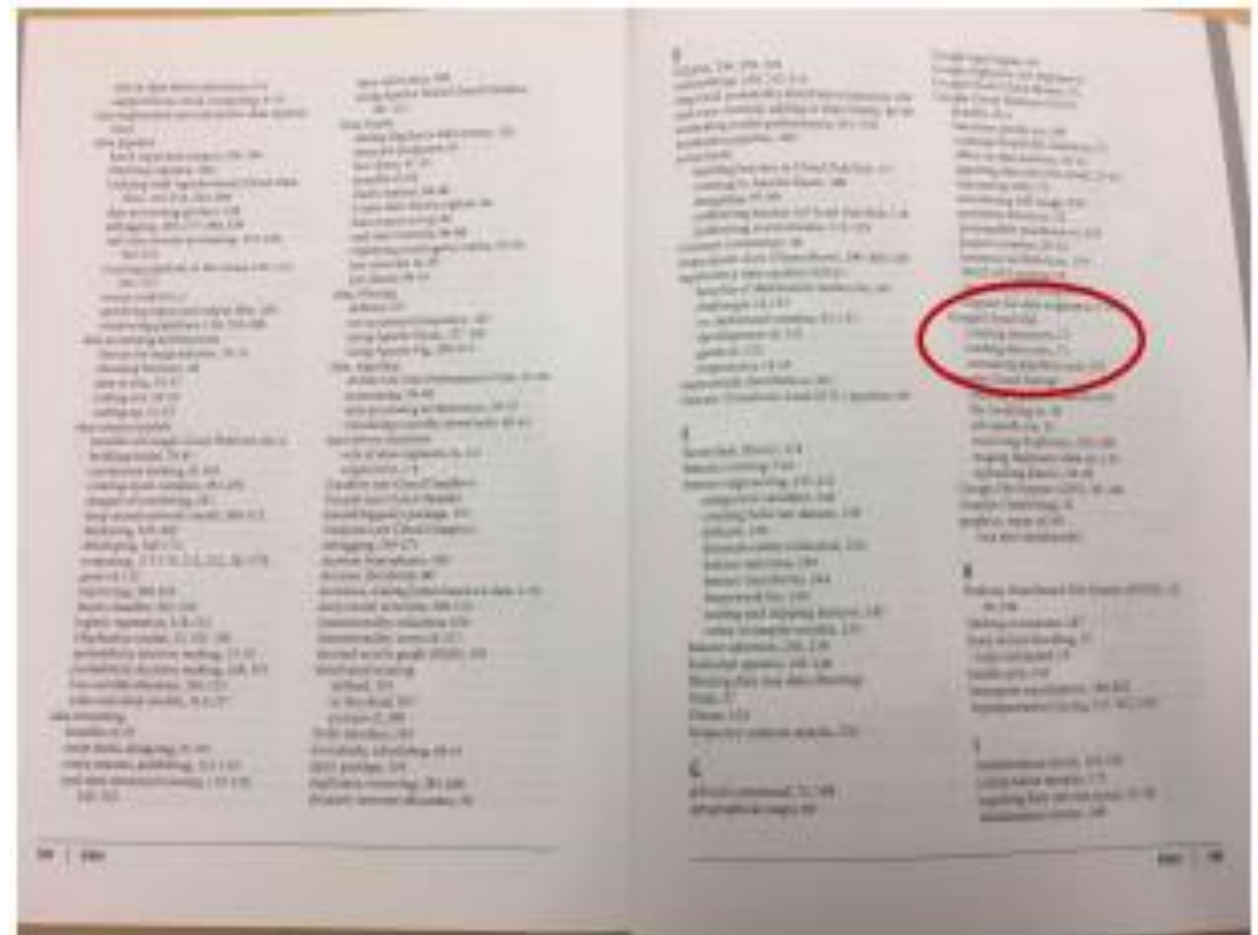


Indexando en un libro

- **Ejemplo:** Encontrar información sobre “Google Cloud SQL” a partir de: *Algún libro*



- Buscar las palabras clave (keywords) en el índice.
- Encontrar las páginas donde aparecen las palabras.
- Leer las páginas para encontrar la información.



Indexando en una base de datos

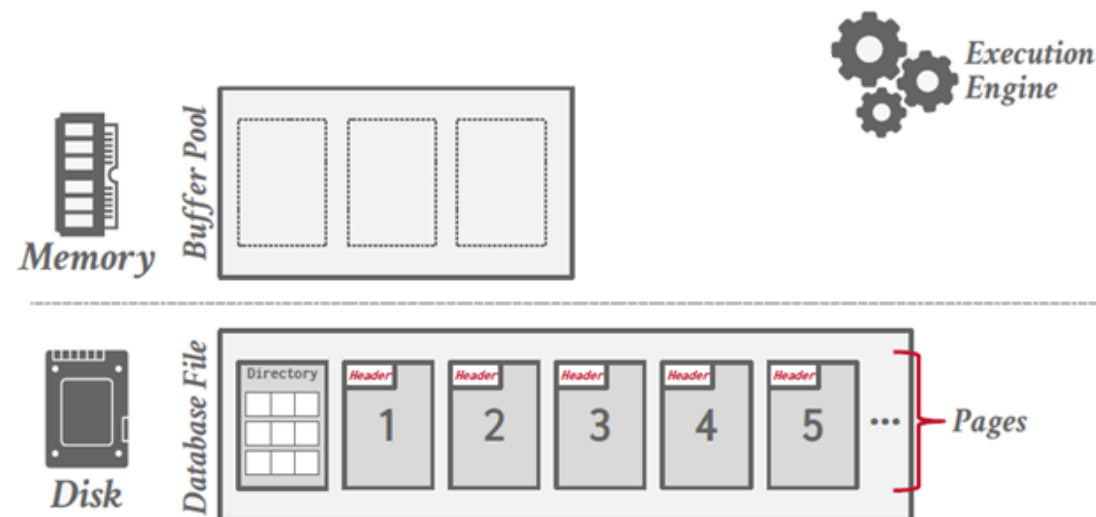
- **Ejemplo:** Encontrar estudiantes de 2º año que hayan cursado menos de 45 créditos.

¿Obtener las 10,000 tuplas y evaluar la condición de la cláusula WHERE sobre cada una?

```
SELECT *  
FROM students  
WHERE year=2 AND credits < 45;
```

Puede haber 10000 tuplas en la relación students.

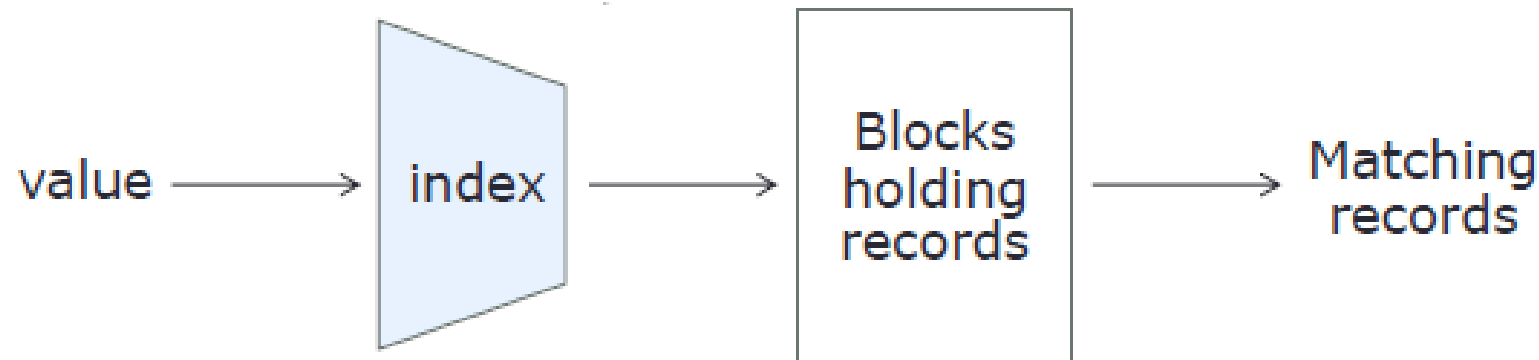
- Determinar en qué bloque de disco se encuentra el registro correspondiente.
- Recuperar (fetch) el bloque de disco.
- Obtener los registros de los estudiantes que cumplen el criterio de búsqueda.



¿Existe una forma de obtener únicamente las tuplas de estudiantes de 2º año y luego evaluar cada una para verificar si los créditos coinciden?

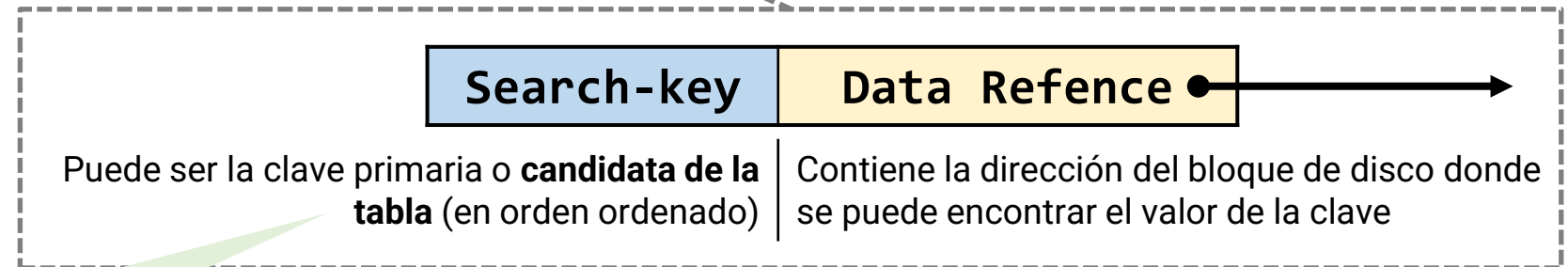
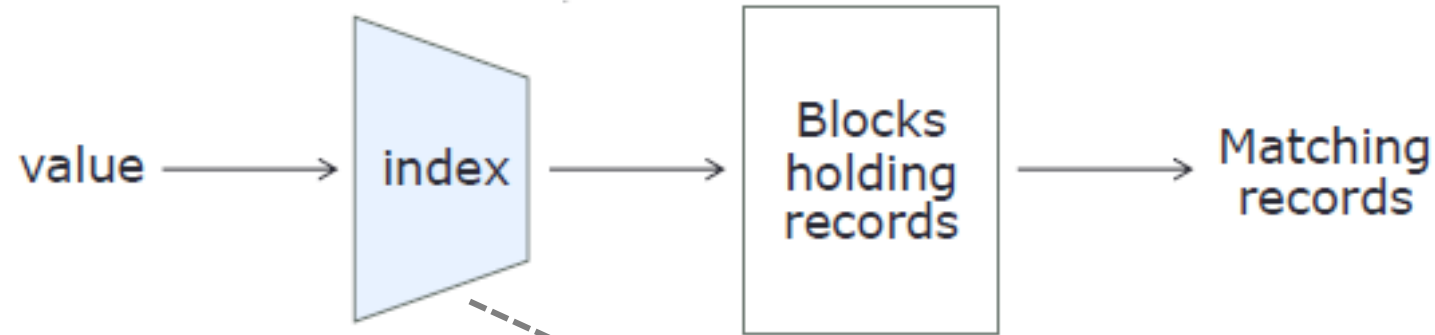
Conceptos básicos

- El **indexado** (*indexing*) es una técnica que consiste en utilizar estructuras de datos auxiliares para permitir que un DBMS localice registros de forma eficiente sin tener que escanear toda una tabla en cada acceso.
- **Clave de búsqueda (Search Key)**: atributo o conjunto de atributos utilizados para buscar registros en un archivo.
- Un archivo de índice (**index file**) consiste en registros (entradas de índice (**index entries**)).



Conceptos básicos

- Un **índice** toma un valor para algún(os) campo(s) y encuentra rápidamente los registros con el valor coincidente



"Search key" (o "key") = Field(s)
sobre cuyos valores se basa el índice.

Estructura general de un Index record

Conceptos básicos

- El índice permite identificar más rápido qué bloque de disco debe leerse, evitando recorrer todos los bloques del archivo.
- Un índice es una **estructura auxiliar** (<search key> --> <Block #>) que mapea una clave de búsqueda a un bloque de disco, permitiendo que el DBMS lea en memoria solo los bloques relevantes mediante operaciones como **ReadBlock(Block #)**



Company(CName, StockPrice, Date, Country)

Company			
CName	Date	Price	Country
AAPL	Oct1	101.23	USA
AAPL	Oct2	102.25	USA
AAPL	Oct3	101.6	USA
GOOG	Oct1	201.8	USA
GOOG	Oct2	201.61	USA
GOOG	Oct3	202.13	USA
Alibaba	Oct1	407.45	China
Alibaba	Oct2	400.23	China

```
SELECT *
FROM Company
WHERE CName like 'AAPL'
```



CName_Index

CName	Block #
AAPL	...
AAPL	...
AAPL	...
GOOG	...
GOOG	...
GOOG	...
GOOG	...
GOOG	...
Alibaba	...
Alibaba	...
Alibaba	...

Company			
CName	Date	Price	Country
AAPL	Oct1	101.23	USA
AAPL	Oct2	102.25	USA
AAPL	Oct3	101.6	USA
GOOG	Oct1	201.8	USA
GOOG	Oct2	201.61	USA
GOOG	Oct3	202.13	USA
Alibaba	Oct1	407.45	China
Alibaba	Oct2	400.23	China

Conceptos básicos

- Un índice sobre uno o más atributos (**search key**) puede acelerar la ejecución de aquellas consultas en las que se especifica un valor, o un rango de valores, para dicho atributo, y puede acelerar ciertas combinaciones (**joins**), dependiendo del plan de ejecución.

```
SELECT *  
FROM Company  
WHERE CName like 'AAPL%'
```

- Por otro lado, todo índice construido sobre uno o más atributos de una relación hace que las inserciones (insertions), eliminaciones (deletions) y actualizaciones (updates) sobre dicha relación sean más complejas y costosas en tiempo, ya que el índice también debe mantenerse actualizado.

```
UPDATE Company  
SET Price = 102.5  
WHERE CName = 'AAPL'
```

Al actualizar, el índice sobre **CName** también debe ser modificado

Company			
CName	Date	Price	Country
AAPL	Oct1	101.23	USA
AAPL	Oct2	102.25	USA
AAPL	Oct3	101.6	USA
GOOG	Oct1	201.8	USA
GOOG	Oct2	201.61	USA
GOOG	Oct3	202.13	USA
Alibaba	Oct1	407.45	China
Alibaba	Oct2	400.23	China

Los diseñadores de bases de datos deben analizar el compromiso (**trade-off**).





¿Cuál técnica de indexación seleccionar?

Aspectos que deben ser considerados:

- **Tipos de acceso (Access types)**
 - Encontrar registros con un valor específico de atributo (**search key**)
 - Encontrar registros con valores de atributo dentro de un rango especificado (**range**)
- **Tiempo de acceso (Access time)**
 - Tiempo necesario para encontrar un ítem de datos particular o un conjunto de ítems de datos
- **Tiempo de inserción (Insertion time)**
 - Tiempo para encontrar la posición de inserción + tiempo para insertar un nuevo ítem de datos + tiempo para actualizar la estructura del índice (**index structure**)
- **Tiempo de eliminación (Deletion time)**
 - Tiempo para encontrar el elemento de datos a eliminar + tiempo para eliminar los datos + tiempo para actualizar la estructura del índice
- **Sobrecarga de espacio (Space overhead)**
 - Espacio ocupado por una estructura de índice (vs. rendimiento)



Mecanismos

- Un **mecanismo de organización de archivos** es la **estrategia que define cómo se almacenan y localizan físicamente los registros en disco**, de manera que el DBMS pueda responder a una consulta sin recorrer el archivo completo. En otras palabras, es la forma en que el índice organiza internamente sus entradas para que la búsqueda sea eficiente.
- La elección del mecanismo determina directamente:
 - **El tipo de consultas** que se pueden responder eficientemente.
 - El **costo** de las operaciones de inserción, eliminación y actualización, ya que el índice también debe mantenerse actualizado.

Tipo de mecanismo	Propósito	¿Qué define?	Ejemplos
ORGANIZACIÓN – ¿Cómo se almacenan los datos?			
Organización de archivos (<i>File Organization</i>)	Definir cómo se almacenan los datos en disco	Estructura física de los registros	Heap files, Sorted files
Estructuras de acceso (<i>Access Structures / Indexes</i>)	Acelerar la localización de datos	Caminos de acceso a los registros	B+ Tree, Hash index
ACCESO – ¿Cómo se recorren o localizan los datos?			
Acceso secuencial (<i>Sequential access</i>)	Procesar todos los datos en orden	Forma de recorrer los datos	File scan, Ordered scan
Acceso directo (<i>Direct access</i>)	Acceder a datos específicos sin recorrer todo el archivo	Estrategia de búsqueda	Index lookup, Hash lookup

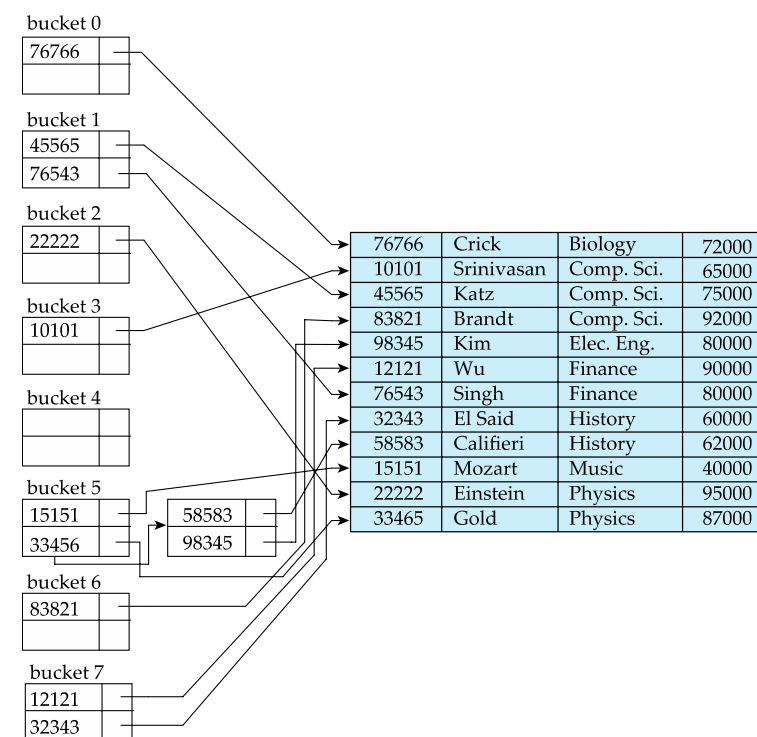
Tipos de Mecanismos de Organización de Archivos

De los mecanismos mencionados, los dos que implementan directamente el **Indexing** son:

- **Organización de archivo secuencial** (soporta igualdad y rango)
- **Organización de archivo hash** (soporta solo igualdad exacta)

Biology		76766	Crick	Biology	72000
Comp. Sci.		10101	Srinivasan	Comp. Sci.	65000
Elec. Eng.		45565	Katz	Comp. Sci.	75000
Finance		83821	Brandt	Comp. Sci.	92000
History		98345	Kim	Elec. Eng.	80000
Music		12121	Wu	Finance	90000
Physics		76543	Singh	Finance	80000
		32343	El Said	History	60000
		58583	Califieri	History	62000
		15151	Mozart	Music	40000
		22222	Einstein	Physics	95000
		33465	Gold	Physics	87000

Archivo de la relación *instructor* ordenado
secuencialmente por *dept_name* como clave



Organización **hash** de la relación *instructor* usando
dept_name como clave

Organización de archivo secuencial

- Índices basados en el ordenamiento de los valores de la clave de búsqueda (**search key**).
- Generalmente rápido.
- Estructura básica / tradicional que utiliza la mayoría de los DBMS.
- Soporta búsquedas por igualdad exacta y por rango de valores.

```
CREATE INDEX idx_instructor_id ON  
instructor (ID);
```

```
SELECT *  
FROM instructor  
WHERE ID = 45565;
```

```
SELECT *  
FROM instructor  
WHERE ID BETWEEN 22222 AND 45565;
```

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Relación *instructor* – sin ordenamiento



10101		10101	Srinivasan	Comp. Sci.	65000	
12121		12121	Wu	Finance	90000	
15151		15151	Mozart	Music	40000	
22222		22222	Einstein	Physics	95000	
32343		32343	El Said	History	60000	
33456		33456	Gold	Physics	87000	
45565		45565	Katz	Comp. Sci.	75000	
58583		58583	Califieri	History	62000	
76543		76543	Singh	Finance	80000	
76766		76766	Crick	Biology	72000	
83821		83821	Brandt	Comp. Sci.	92000	
98345		98345	Kim	Elec. Eng.	80000	

Archivo de la relación *instructor* ordenado
secuencialmente por ID

Organización de archivo hash

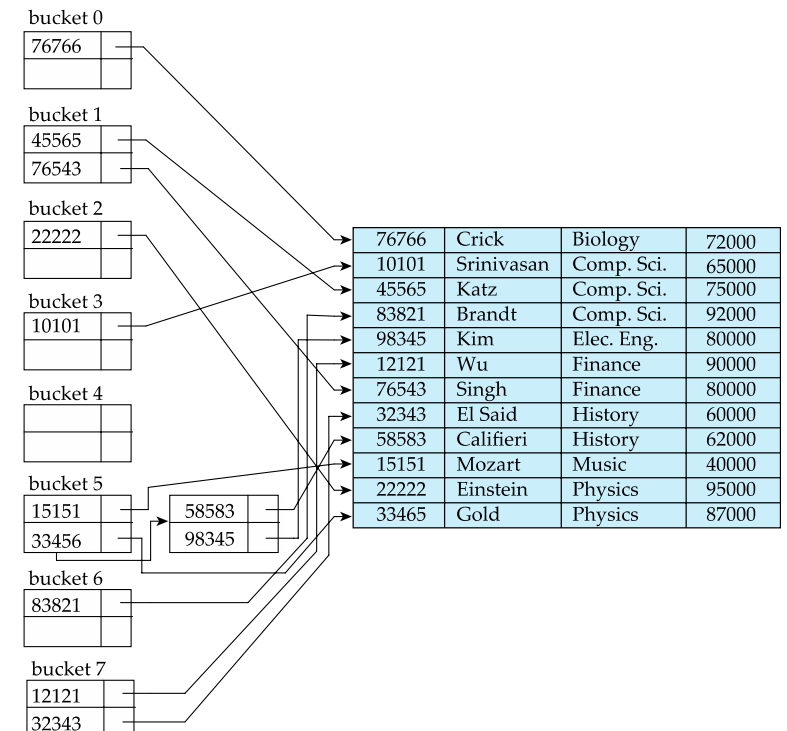
- Índices basados en una distribución uniforme de valores a través de un rango de cubetas (**buckets**).
- La función de dispersión (**hash function**) determina la cubeta asignada a cada registro.
- Soporta únicamente búsquedas por igualdad exacta.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Relación *instructor* – sin ordenamiento

Nota: El mismo archivo base puede organizarse de forma hash en lugar de secuencial

```
CREATE INDEX idx_instructor_dept
ON instructor(dept_name)
USING HASH;
```



Organización hash de la relación *instructor* usando *dept_name* como clave

Ejemplo 1 – Costo sin índice

Una tabla de Estudiantes tiene 1000000 de registros almacenados en 200000 bloques de disco. Teniendo en cuenta esta situación, responda las siguientes preguntas:

1. ¿Cuántos registros por bloque?
2. Si cada lectura de disco toma 10 ms, ¿cuántos segundos tarda un full scan sobre 200.000 bloques?

1. $N_B = ?$

Solución:

• **Datos:**

- $N = 1000000$
- $B = 200000$
- $T_{read} = 10 \text{ ms/bloque}$

$$N_B = \frac{N}{B} = \frac{1000000}{200000} = 5 \text{ reg/bloque}$$

2. $T_{FS} = ?$

$$T_{FS} = 200000 \text{ bloques} \times \frac{10 \text{ ms}}{1 \text{ bloque}} = 2000000 \text{ ms}$$

$$T_{FS} = 2000000 \text{ ms} \times \frac{1 \text{ s}}{1000 \text{ ms}} \times \frac{1 \text{ min}}{60 \text{ s}} = 33.33 \text{ min}$$



Ejemplo 1 – Costo sin índice

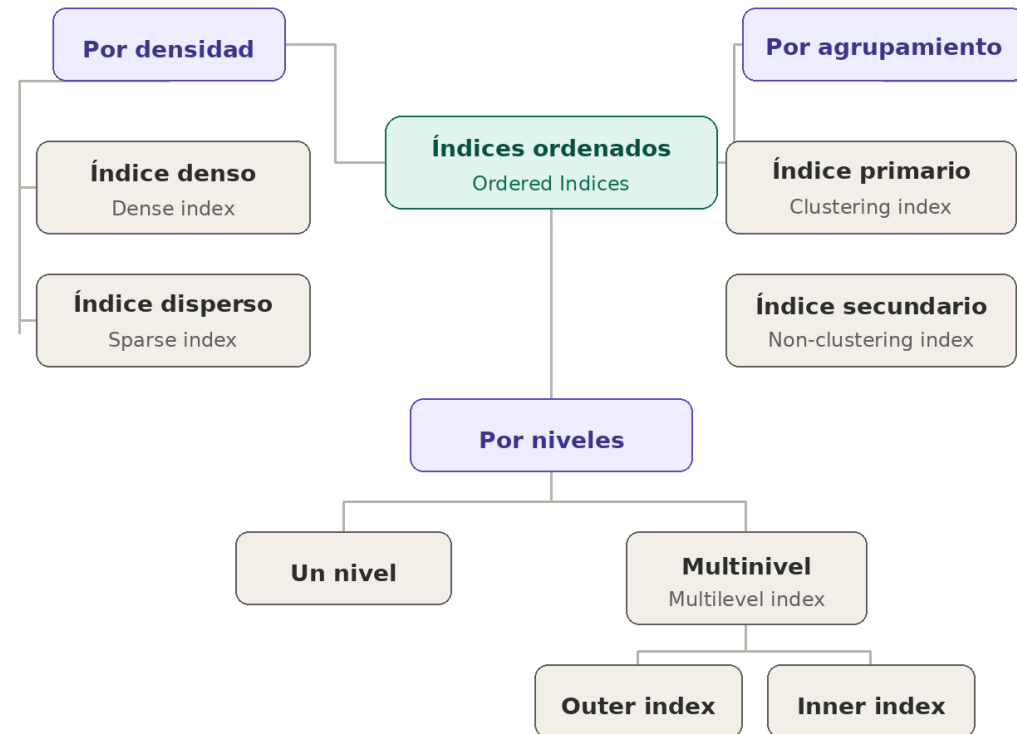
Sin índice – 15 primeros registros (1 por bloque, 5 registros/bloque)

ID	Nombre	Carrera	Bloque
101	Ana	Sistemas	B1
102	Luis	Matemáticas	
103	Sara	Sistemas	
104	Pedro	Física	
105	Marta	Sistemas	
106	Juan	Química	B2
107	Sofía	Matemáticas	
108	Carlos	Física	
109	Laura	Sistemas	
110	Diego	Química	
111	Valeria	Matemáticas	B3
112	Andrés	Sistemas	
113	Camila	Física	
114	Tomás	Química	
115	Isabel	Sistemas	



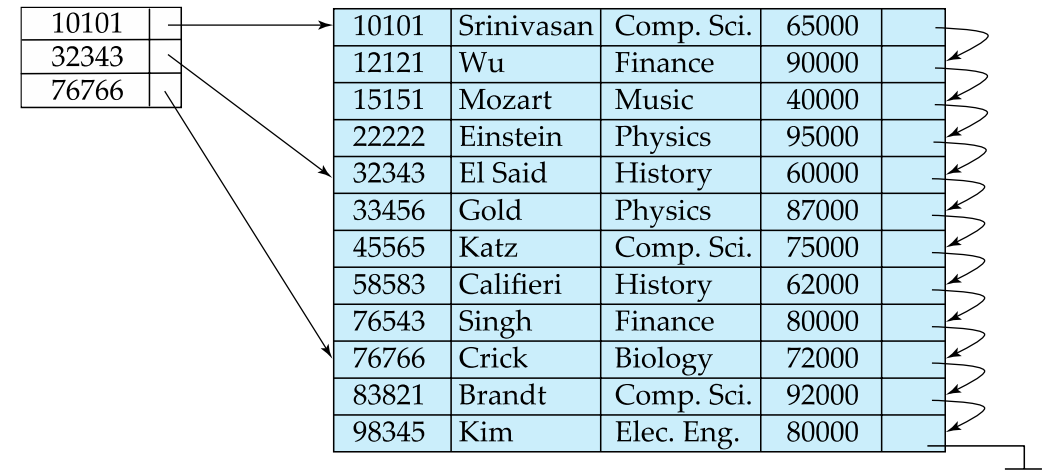
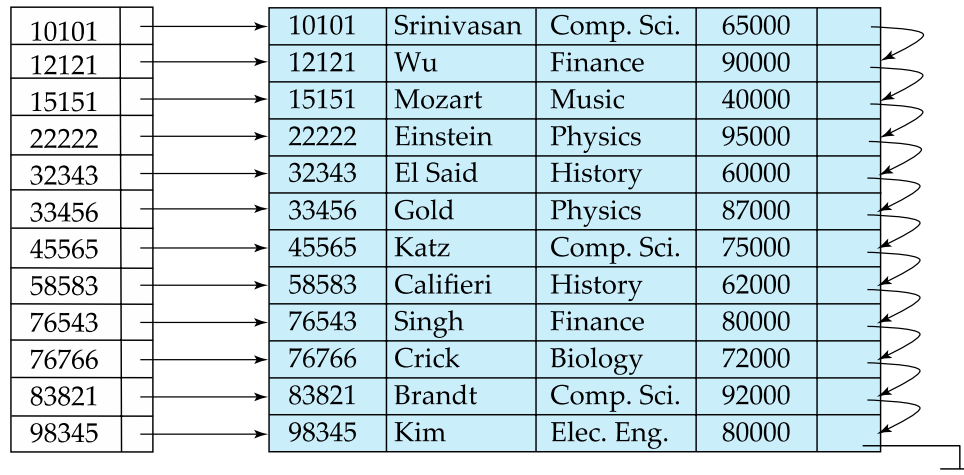
De los mecanismos a los índices ordenados

De los dos mecanismos vistos, la organización secuencial se implementa a través de una familia de técnicas conocidas como índices ordenados (**ordered indices**). A continuación exploraremos esta familia.



¿Qué tan denso es un índice?

- Cuando se construye un índice, una decisión fundamental es **cuántas entradas** tendrá: ¿una por cada registro del archivo, o solo algunas?
- A esta característica se le denomina **densidad del índice** y determina directamente el equilibrio entre velocidad de búsqueda y espacio ocupado.
- Existen dos tipos:
 - **Índice denso (Dense index)**: existe una entrada en el índice por cada registro del archivo (mapeo 1:1).
 - **Índice disperso (Sparse index)**: el índice contiene entradas solo para algunos valores de la clave de búsqueda.



La densidad no es una propiedad menor: define cuánto espacio ocupa el índice y qué tan rápido puede localizar un registro.

Índice denso (Dense index)

- En un índice denso existe **una entrada de índice por cada valor de clave de búsqueda** presente en el archivo.
- Cada entrada contiene el valor de la clave y un puntero (***pointer***) al registro correspondiente en el archivo.
- Permite localizar **directamente** cualquier registro sin necesidad de recorrer el archivo.
- Es más rápido en las búsquedas, pero ocupa **más espacio** que un índice disperso, ya que debe mantener una entrada por cada registro.

10101	→	10101	Srinivasan	Comp. Sci.	65000	→
12121	→	12121	Wu	Finance	90000	→
15151	→	15151	Mozart	Music	40000	→
22222	→	22222	Einstein	Physics	95000	→
32343	→	32343	El Said	History	60000	→
33456	→	33456	Gold	Physics	87000	→
45565	→	45565	Katz	Comp. Sci.	75000	→
58583	→	58583	Califieri	History	62000	→
76543	→	76543	Singh	Finance	80000	→
76766	→	76766	Crick	Biology	72000	→
83821	→	83821	Brandt	Comp. Sci.	92000	→
98345	→	98345	Kim	Elec. Eng.	80000	→

Índice denso (Dense index)

- El comportamiento de un índice denso depende de si la clave de búsqueda es única o no.

```
-- Índice sobre clave primaria
CREATE INDEX idx_instructor_id
ON instructor(ID);
```

10101	10101	Srinivasan	Comp. Sci.	65000
12121	12121	Wu	Finance	90000
15151	15151	Mozart	Music	40000
22222	22222	Einstein	Physics	95000
32343	32343	El Said	History	60000
33456	33456	Gold	Physics	87000
45565	45565	Katz	Comp. Sci.	75000
58583	58583	Califieri	History	62000
76543	76543	Singh	Finance	80000
76766	76766	Crick	Biology	72000
83821	83821	Brandt	Comp. Sci.	92000
98345	98345	Kim	Elec. Eng.	80000

10101	-> r1
12121	-> r2
15151	-> r3
22222	-> r4
32343	-> r5
...	...

```
-- Índice sobre atributo no único
CREATE INDEX idx_instructor_dept
ON instructor(dept_name);
```

Biology	76766	Crick	Biology	72000
Comp. Sci.	10101	Srinivasan	Comp. Sci.	65000
Elec. Eng.	45565	Katz	Comp. Sci.	75000
Finance	83821	Brandt	Comp. Sci.	92000
History	98345	Kim	Elec. Eng.	80000
Music	12121	Wu	Finance	90000
Physics	76543	Singh	Finance	80000
	32343	El Said	History	60000
	58583	Califieri	History	62000
	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	33465	Gold	Physics	87000

Biology	-> r1
Comp. Sci.	-> r2
Comp. Sci.	-> r3
Comp. Sci.	-> r4
Elec. Eng.	-> r5
...	...

Índice denso (Dense index)

- **Ejemplo 1:** La clave de búsqueda (*search key*) es una clave primaria (*primary key*).
 - Se crea un registro por cada valor de clave de búsqueda.
 - Se necesita más espacio para almacenar los registros del índice (*index records*).

```
SELECT * FROM instructor WHERE ID = 58583;
```

Si el archivo ya está ordenado por la clave primaria (*primary key*), entonces construir sobre él un índice primario denso puede ser redundante, porque agrega una entrada por cada registro y consume espacio adicional sin aportar tanta ventaja frente a otras alternativas, como un índice disperso.

10101		10101	Srinivasan	Comp. Sci.	65000	
12121		12121	Wu	Finance	90000	
15151		15151	Mozart	Music	40000	
22222		22222	Einstein	Physics	95000	
32343		32343	El Said	History	60000	
33456		33456	Gold	Physics	87000	
45565		45565	Katz	Comp. Sci.	75000	
58583		58583	Califieri	History	62000	
76543		76543	Singh	Finance	80000	
76766		76766	Crick	Biology	72000	
83821		83821	Brandt	Comp. Sci.	92000	
98345		98345	Kim	Elec. Eng.	80000	

El archivo **Instructor** está ordenado por el ID del instructor (cada valor de clave de búsqueda).

Apunta al registro real en disco

Cuanto más registros tenga el archivo, más grande será el índice denso — una entrada por cada uno.

Índice denso (Dense index)

- **Ejemplo 2:** La clave de búsqueda (search key) no es una clave primaria.
 - Cuando la **clave de búsqueda (search key)** no es única, pueden existir **múltiples registros** con el mismo valor.
 - En este caso, el índice mantiene una entrada por cada valor, pero el acceso a los registros ocurre de forma secuencial.

Cuando la clave no es única, el índice no devuelve un solo registro, sino un punto de entrada a una secuencia de registros relacionados.

```
SELECT *  
FROM instructor  
WHERE dept_name = 'History';
```

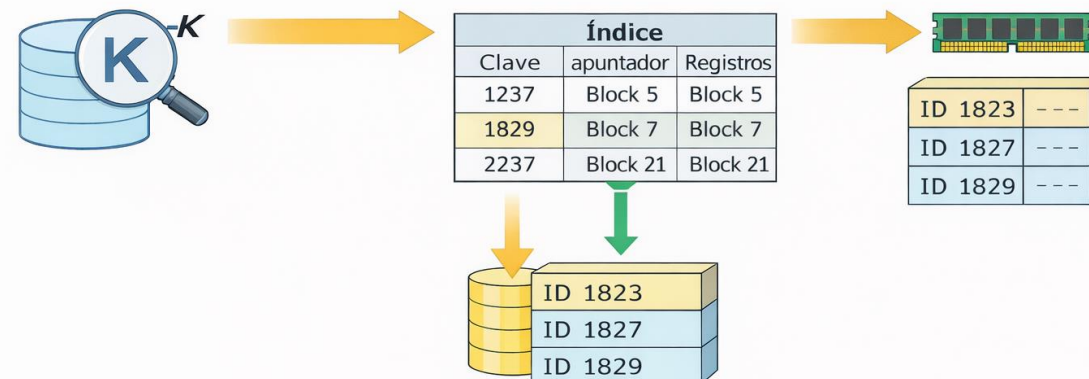
El archivo **Instructor** está ordenado por la clave de búsqueda **dept_name**.

En la búsqueda se sigue el puntero directamente hasta el primer registro y luego se continua siguiendo el puntero en ese registro para ubicar el siguiente registro en orden de la clave de búsqueda, hasta encontrar el registro deseado

Biology		76766	Crick	Biology	72000	
Comp. Sci.		10101	Srinivasan	Comp. Sci.	65000	
Elec. Eng.		45565	Katz	Comp. Sci.	75000	
Finance		83821	Brandt	Comp. Sci.	92000	
History		98345	Kim	Elec. Eng.	80000	
Music		12121	Wu	Finance	90000	
Physics		76543	Singh	Finance	80000	
		32343	El Said	History	60000	
		58583	Califieri	History	62000	
		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		33465	Gold	Physics	87000	

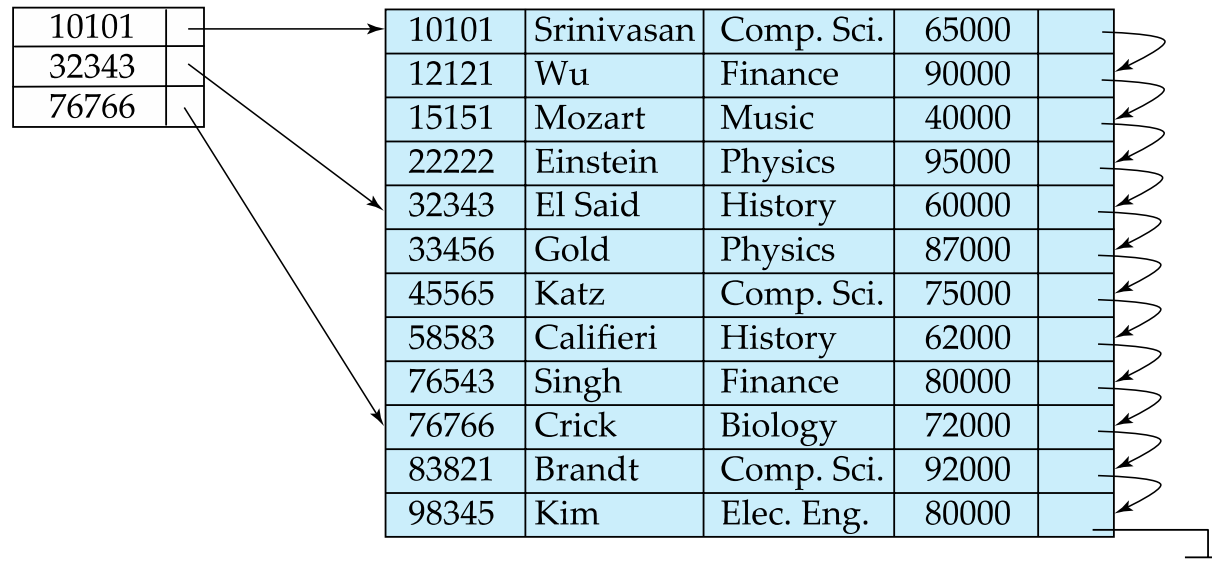
Índice denso: Búsqueda (Lookup)

- Dada una clave de búsqueda (search key) $\langle K \rangle$, el índice es recorrido.
 - Cuando se encuentra $\langle K \rangle$, se sigue el puntero (pointer) asociado y se lee el bloque correspondiente mediante: **ReadBlock(Block #)**
- Cuando el índice se construye sobre una clave no primaria (non-primary key):
 - Se localiza el primer valor (mínimo).
 - Luego se leen bloques consecutivos en memoria hasta agotar los registros que coinciden.
- El índice se mantiene usualmente en **memoria principal (main memory)**. Por lo tanto, la búsqueda requiere típicamente **una sola operación de E/S de disco (disk I/O)**.
- La resolución de consultas mediante índices densos es **eficiente**.
 - El índice está ordenado (*sorted*), se puede utilizar **búsqueda binaria**:
 - Si existen n claves de búsqueda, se requieren a lo sumo $\log_2 n$ pasos para localizar una clave dada.



Índice disperso (Sparse index)

- En un índice disperso el índice **contiene entradas solo para algunos valores** de la clave de búsqueda, no para todos los registros.
- Solo es aplicable cuando los registros están **ordenados secuencialmente** por la clave de búsqueda.
- Ocupa menos espacio y tiene menor costo de mantenimiento que el índice denso, pero es más lento en la localización de registros individuales.



```
CREATE INDEX idx_instructor_id  
ON instructor(ID);
```

Dense vs Sparse es una decisión de implementación del DBMS, no una instrucción del usuario en SQL

El índice apunta al inicio de un bloque, por lo que es necesario buscar dentro del bloque

Índice disperso: Búsqueda (Lookup)

- **Búsqueda:** Para localizar un registro con valor de clave $\langle K \rangle$, el proceso es:
 1. Encontrar la entrada de índice con el mayor valor de clave $\langle K \rangle$.
 2. Buscar secuencialmente dentro del bloque a partir de ese punto.
- **El índice usualmente se mantiene en memoria principal.** Por lo tanto, se requiere realizar una **operación de E/S a disco** durante la búsqueda.
- **Eficiente en espacio**, pero puede requerir **más tiempo de cómputo** debido a dos búsquedas:

```
-- Para responder eficientemente a:  
SELECT * FROM instructor WHERE ID = 45565;  
-- El DBMS encuentra la entrada más cercana  
-- y busca secuencialmente desde allí.
```

Se inicia en la clave más grande menor o igual al valor buscado, y a partir de ahí se realiza una búsqueda lineal.

10101		10101	Srinivasan	Comp. Sci.	65000	
32343		12121	Wu	Finance	90000	
76766		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		32343	El Said	History	60000	
		33456	Gold	Physics	87000	
		45565	Katz	Comp. Sci.	75000	
		58583	Califieri	History	62000	
		76543	Singh	Finance	80000	
		76766	Crick	Biology	72000	
		83821	Brandt	Comp. Sci.	92000	
		98345	Kim	Elec. Eng.	80000	

Compromiso adecuado: un índice disperso con una entrada por cada bloque del archivo ofrece el mejor balance entre espacio y velocidad.

¿Denso o disperso? El compromiso (trade-off)

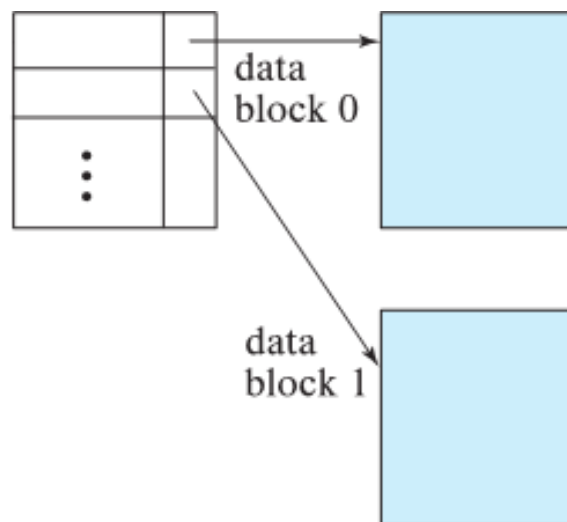
- Ninguno de los dos tipos es universalmente mejor: la elección depende del contexto de uso.

	Índice denso (Dense)	Índice disperso (Sparse)
Entradas	Una por cada registro	Una por cada bloque
Velocidad de búsqueda	Más rápido	Más lento
Espacio ocupado	Mayor	Menor
Costo de actualización	Mayor	Menor
Requisito	Ninguno	Archivo ordenado
Uso típico	Índices secundarios	Índices de agrupamiento

¿Denso o disperso? El compromiso (trade-off)

- **Regla práctica (Libro Silberschatz):**

- Para un **índice de agrupamiento (*clustered*)**: usar índice disperso con una entrada por bloque.



- Para un **índice no agrupado (*unclustered*)**: usar índice disperso sobre índice denso (multinivel).

La densidad del índice no se elige de forma aislada: siempre se decide en función del tipo de agrupamiento y del patrón de consultas del sistema.

Ejemplo 2 – Costo con índice denso

Una tabla de **Estudiantes** tiene 1,000,000 de registros almacenados en 200,000 bloques de disco. Las columnas de la tabla son: **ID**, **Nombre** y **Carrera** (Sistemas, Matemáticas, Física o Química). Cada lectura de disco toma **10 ms**, y cada bloque del índice puede almacenar **100 entradas**. Teniendo en cuenta esta situación, responda las siguientes preguntas:

1. Si se construye un índice **denso** basado en el ID del estudiante, ¿cuántas entradas tiene este índice?
2. Si el índice denso se construye sobre el atributo **Carrera**, ¿cuántas entradas tiene?
3. Asuma que el índice está almacenado en disco. ¿Cuántos bloques ocupa el índice denso sobre ID? ¿Cuál es el costo en I/O y en tiempo para recuperar un único registro usando este índice con búsqueda binaria?
4. Sin ningún índice, ¿cuál sería el costo en tiempo de recuperar un registro haciendo un escaneo completo del archivo? ¿Qué concluye al comparar este resultado con el punto anterior?

Ejemplo 2 – Costo con índice denso

1. **Índice denso sobre ID** (ID, puntero $\rightarrow$): En un índice denso hay una entrada por cada registro de la tabla.

$$N_{Index} = N = 1000000$$

2. **Índice denso sobre Carrera** (Carrera, puntero $\rightarrow$): Aunque Carrera solo tenga 4 valores distintos en un índice denso sigue habiendo una entrada por registro.

$$N_I = N = 1000000$$

3. **Parte 1:** Cantidad de bloques ocupadas por el índice denso:

$$B_I = \frac{N}{N_{B_I}} = \frac{1000000}{100} = 10000 \text{ bloques}$$

Ejemplo 2 – Costo con índice denso

3. **Parte 2. Costo total de acceso:** El costo total se debe al costo de la búsqueda binaria (para encontrar el índice) y el costo de acceso al registro de interés una vez este ha encontrado.

$$I/O = I/O_{search} + I/O_{reg}$$

$$I/O_{search} = \lceil \log_2 B \rceil = \lceil \log_2 10000 \rceil = \lceil 13.28 \rceil = 14$$

$$I/O_{reg} = 1$$

$$I/O = 14 + 1 = 15$$

$$T_{I/O} = 15 \times 10 \text{ ms} = 150 \text{ ms}$$

Ejemplo 2 – Costo con índice denso

Índice denso – 15 entradas (1 por registro, 5 registros por bloque)

ID	Puntero		ID	Nombre	Carrera	Bloque
101	→ r1	→	101	Ana	Sistemas	B1
102	→ r2	→	102	Luis	Matemáticas	
103	→ r3	→	103	Sara	Sistemas	
104	→ r4	→	104	Pedro	Física	
105	→ r5	→	105	Marta	Sistemas	
106	→ r6	→	106	Juan	Química	B2
107	→ r7	→	107	Sofía	Matemáticas	
108	→ r8	→	108	Carlos	Física	
109	→ r9	→	109	Laura	Sistemas	
110	→ r10	→	110	Diego	Química	
111	→ r11	→	111	Valeria	Matemáticas	B3
112	→ r12	→	112	Andrés	Sistemas	
113	→ r13	→	113	Camila	Física	
114	→ r14	→	114	Tomás	Química	
115	→ r15	→	115	Isabel	Sistemas	

Ejemplo 2 – Costo con índice denso

4. **Comparación:** La siguiente tabla resume los resultados:

	Entradas	Bloques del índice	Costo búsqueda	Costo tiempo
Índice denso	1000000	10000	15 I/Os	150ms
Sin índice	—	200000	200000 I/Os	~33 min

Ejemplo 3 – Costo con índice disperso

Usando la misma tabla de Estudiantes, ahora se construye un **índice disperso** sobre el ID, con una entrada por cada bloque del archivo.

1. ¿Cuántas entradas tiene este índice?
2. ¿Cuántos bloques ocupa el índice si cada bloque almacena 100 entradas?
3. ¿Cuál es el costo en I/O para recuperar el registro con ID = 45565?
4. Compara el espacio ocupado por el índice denso vs el disperso. ¿Qué concluyes?

Ejemplo 3 – Costo con índice disperso

1. **Índice disperso sobre ID (ID, puntero →):** En un índice disperso hay una entrada por bloque del archivo de datos, no una por registro.

$$N_{Index} = B = 200000$$

Comparado con el índice denso (1000000 entradas), el índice denso tiene 5 veces menos entradas.

2. Cantidad de bloques ocupadas por el índice denso:

$$B_I = \frac{N}{N_{B_I}} = \frac{200000}{100} = 2000 \text{ bloques}$$

Ejemplo 3 – Costo con índice disperso

3. Costo I/O para recuperar el **ID = 45565**.

$$I/O = I/O_{search} + I/O_{reg}$$

$$I/O_{search} = \lceil \log_2 B \rceil = \lceil \log_2 2000 \rceil = \lceil 10.96 \rceil = 11$$

$$I/O_{reg} = 1$$

$$I/O = 11 + 1 = 12$$

$$T_{I/O} = 12 \times 10 \text{ ms} = 120 \text{ ms}$$

Ejemplo 3 – Costo con índice disperso

Índice disperso – 3 entradas (1 por bloque, 5 registros/bloque)

ID	Puntero		ID	Nombre	Carrera	Bloque
101	→ b1	→	101	Ana	Sistemas	B1
106	→ b2	→	102	Luis	Matemáticas	
111	→ b3	→	103	Sara	Sistemas	
		→	104	Pedro	Física	
		→	105	Marta	Sistemas	
		→	106	Juan	Química	B2
			107	Sofía	Matemáticas	
			108	Carlos	Física	
			109	Laura	Sistemas	
			110	Diego	Química	
			111	Valeria	Matemáticas	B3
			112	Andrés	Sistemas	
			113	Camila	Física	
			114	Tomás	Química	
			115	Isabel	Sistemas	

Ejemplo 3 – Costo con índice disperso

4. **Comparación:** La siguiente tabla resume los resultados:

	Entradas	Bloques del índice	Costo búsqueda	Costo tiempo
Índice denso	1000000	10000	15 I/Os	150ms
Índice disperso	200000	2000	12 I/Os	120ms
Sin índice	—	200000	200000 I/Os	~33 min

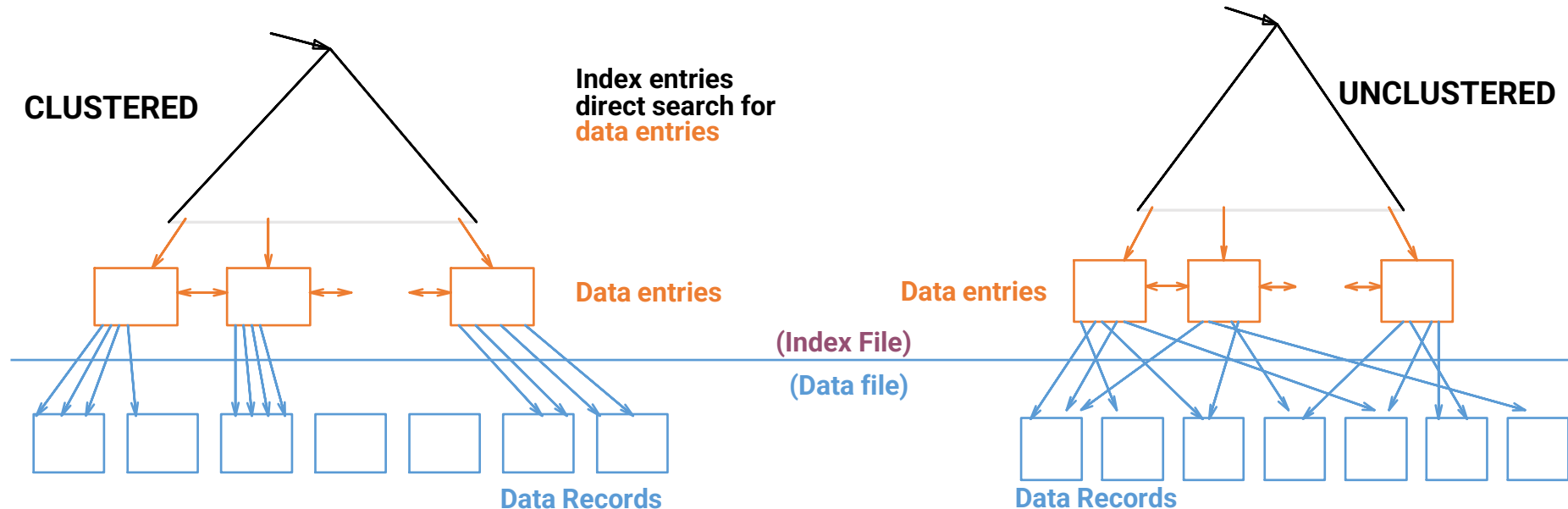
Conclusiones:

- El índice disperso ocupa 5 veces menos espacio que el denso (2,000 bloques **vs** 10,000 bloques).
- En este caso el índice disperso es ligeramente más rápido (12 I/Os **vs** 15 I/Os), porque su índice tiene menos bloques y la búsqueda binaria requiere menos pasos.
- Sin embargo, el índice disperso requiere que el archivo esté ordenado por la clave de búsqueda — sin ese orden, no puede funcionar.
- Además, el índice disperso obliga a una búsqueda secuencial dentro del bloque al final, lo que agrega cómputo extra sin costo de disco adicional.
- El índice denso garantiza acceso directo al registro exacto sin ninguna búsqueda adicional, a cambio de mayor espacio.
- Ninguno es universalmente mejor: la elección depende del espacio disponible, el patrón de consultas y si el archivo está ordenado.



¿El índice sigue el orden del archivo?

- Una segunda dimensión fundamental de los índices ordenados es su relación con el **orden físico del archivo**.
- **La pregunta clave es:** ¿el orden de las entradas del índice coincide con el orden en que los registros están almacenados físicamente en disco?

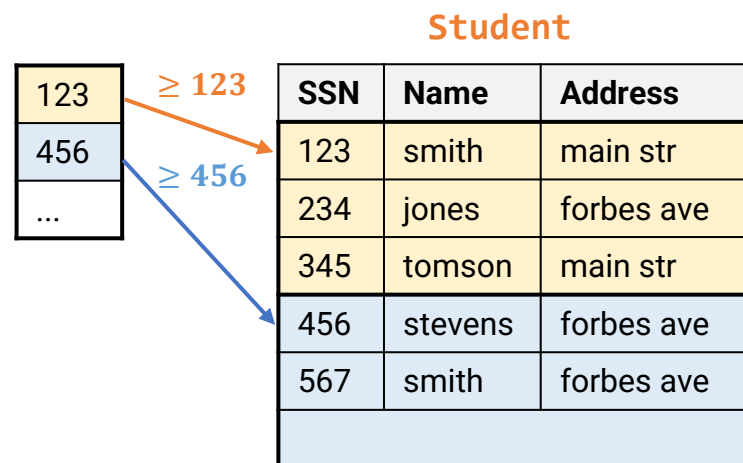


Esta distinción no es menor: define si el DBMS puede aprovechar la proximidad física de los registros para hacer búsquedas más eficientes.

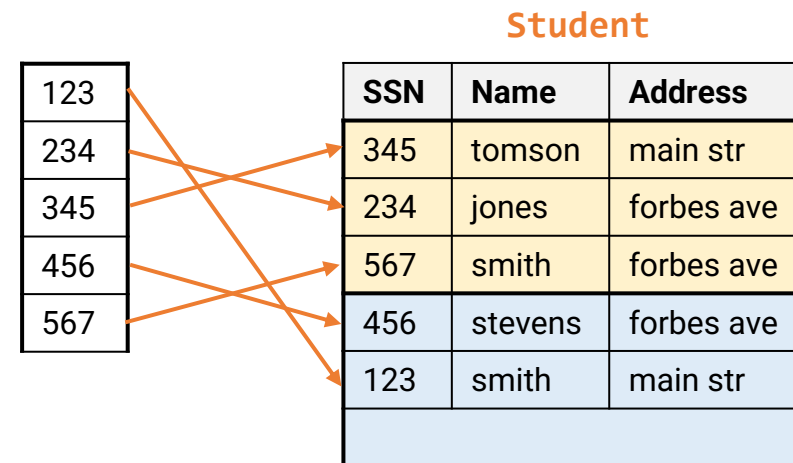
¿El índice sigue el orden del archivo?

- **Pregunta:** ¿el orden de las entradas del índice coincide con el orden en que los registros están almacenados físicamente en disco?
- El índice se clasifica en uno de dos tipos:
 - **Índice de agrupamiento (Clustering index):** el orden del índice coincide con el orden físico del archivo. También llamado índice primario (*primary index*).
 - **Índice secundario (Secondary index):** el orden del índice es diferente al orden físico del archivo. También llamado índice no agrupado (*non-clustering index*).

Clustering / Índice disperso sobre SSN



No Clustering / Índice denso sobre SSN



Índice de agrupamiento (Clustering index)

- En un índice de agrupamiento, el orden de las entradas del índice **coincide con el orden físico** en que los registros están almacenados en el archivo.
- También denominado **índice primario** (*primary index*).
 - La clave de búsqueda del índice primario es usualmente, aunque **no necesariamente**, la clave primaria (*primary key*) de la relación.
 - Al estar alineado con el orden físico del archivo, permite recorrer rangos de registros de forma **secuencial y eficiente**.

Dense index — una
entrada por registro

Archivo ordenado por ID

10101	→	10101	Srinivasan	Comp. Sci.	65000	→
12121	→	12121	Wu	Finance	90000	→
15151	→	15151	Mozart	Music	40000	→
22222	→	22222	Einstein	Physics	95000	→
32343	→	32343	El Said	History	60000	→
33456	→	33456	Gold	Physics	87000	→
45565	→	45565	Katz	Comp. Sci.	75000	→
58583	→	58583	Califieri	History	62000	→
76543	→	76543	Singh	Finance	80000	→
76766	→	76766	Crick	Biology	72000	→
83821	→	83821	Brandt	Comp. Sci.	92000	→
98345	→	98345	Kim	Elec. Eng.	80000	→

Acceso directo a cualquier registro en una I/O

Una entrada por valor
único de dept_name

Registros del mismo dept agrupados físicamente

Biology	→	76766	Crick	Biology	72000	→
Comp. Sci.	→	10101	Srinivasan	Comp. Sci.	65000	→
Elec. Eng.	→	45565	Katz	Comp. Sci.	75000	→
Finance	→	83821	Brandt	Comp. Sci.	92000	→
History	→	98345	Kim	Elec. Eng.	80000	→
Music	→	12121	Wu	Finance	90000	→
Physics	→	76543	Singh	Finance	80000	→
	→	32343	El Said	History	60000	→
	→	58583	Califieri	History	62000	→
	→	15151	Mozart	Music	40000	→
	→	22222	Einstein	Physics	95000	→
	→	33465	Gold	Physics	87000	→

Desde el punto de entrada se recorre secuencialmente el grupo

Ambos son clustering indexes — el orden físico del archivo coincide con el orden del índice. La diferencia está en la unicidad de la search key

Índice de agrupamiento (Clustering index)

- ¿Qué gana el DBMS con el orden físico del **clustering index**?

```
-- Aprovecha al máximo el clustering index:  
SELECT * FROM instructor  
WHERE ID BETWEEN 22222 AND 45565;  
-- Los registros están físicamente contiguos en disco.
```

BETWEEN aprovecha el orden físico — el DBMS entra por 22222 y lee bloques consecutivos hasta 45565

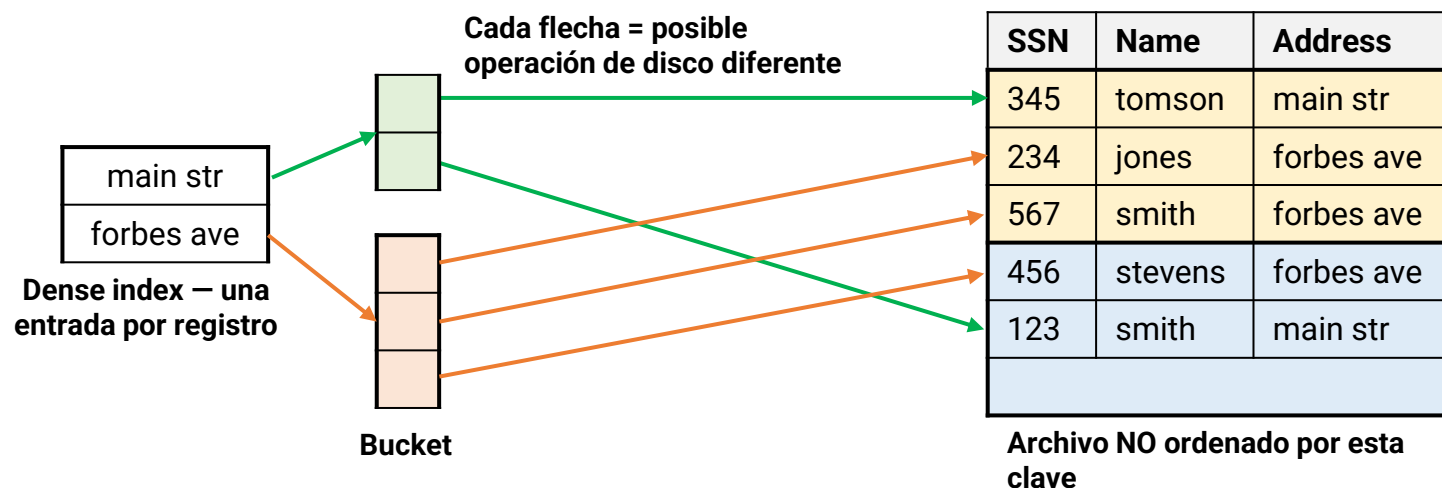
10101	→	10101	Srinivasan	Comp. Sci.	65000	
12121	→	12121	Wu	Finance	90000	
15151	→	15151	Mozart	Music	40000	
22222	→	22222	Einstein	Physics	95000	
32343	→	32343	El Said	History	60000	
33456	→	33456	Gold	Physics	87000	
45565	→	45565	Katz	Comp. Sci.	75000	
58583	→	58583	Califieri	History	62000	
76543	→	76543	Singh	Finance	80000	
76766	→	76766	Crick	Biology	72000	
83821	→	83821	Brandt	Comp. Sci.	92000	
98345	→	98345	Kim	Elec. Eng.	80000	

Registros físicamente contiguos en disco

Un índice de agrupamiento es como un índice de libro donde los capítulos están en el mismo orden en que aparecen las páginas — navegar es natural y eficiente.

Índice secundario (Secondary index)

- En un índice secundario, el orden del índice **es diferente al orden físico** del archivo.
 - También **denominado índice no agrupado** (*non-clustering index*)
 - Los registros apuntados no están físicamente contiguos — cada acceso puede requerir una E/S diferente.
- Los índices secundarios **deben ser densos**
 - El archivo no está ordenado por esta clave → se necesita una entrada por cada registro
 - Cada entrada apunta a una **cubeta** (*bucket*) con punteros a todos los registros con ese valor



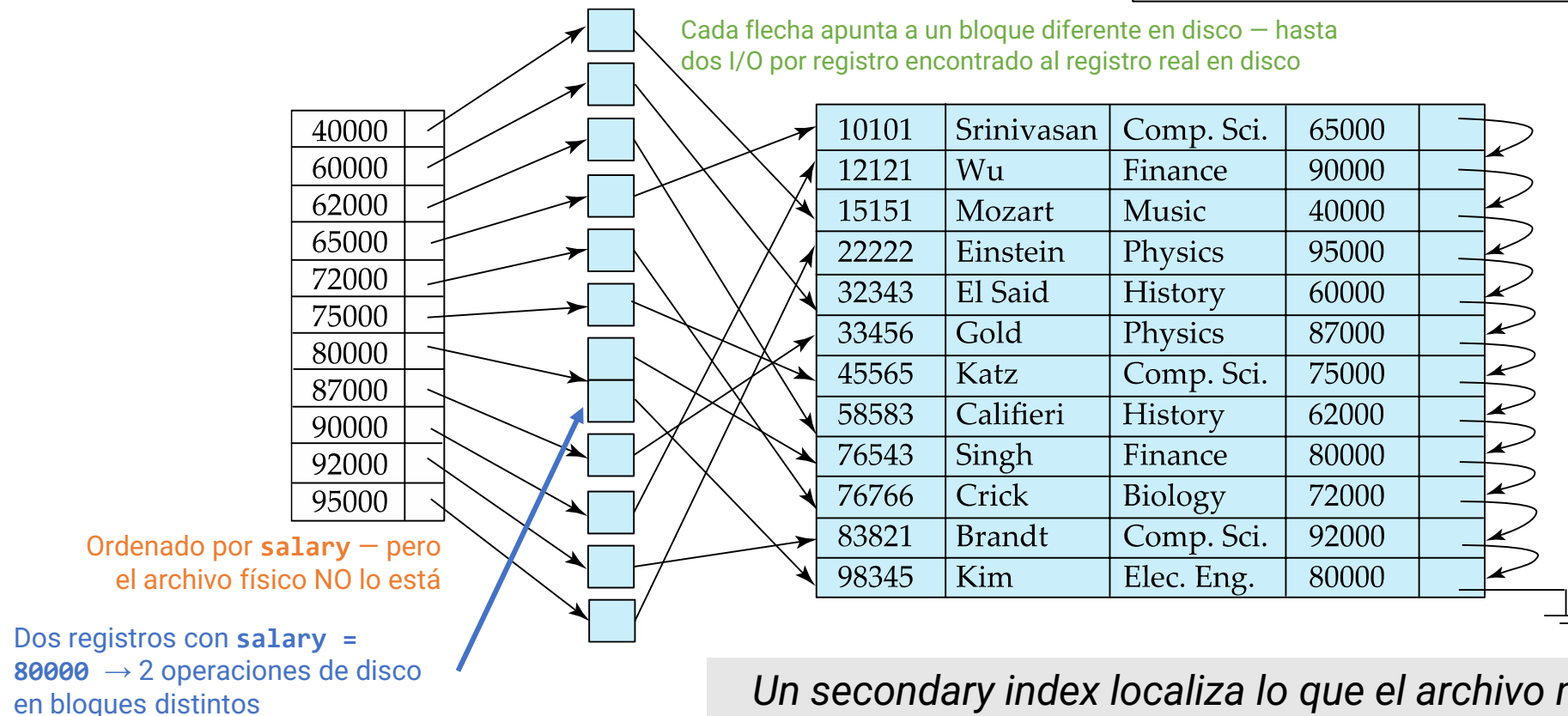
Un índice secundario es poderoso pero costoso: permite buscar por cualquier atributo, pero cada acceso puede implicar una operación de disco diferente.

Índice secundario (Secondary index)

- ¿Qué paga el DBMS por buscar por un atributo no ordenado?

El índice anterior funcionaba con **address** (2 valores).
¿Qué pasa con 12 registros y salarios dispersos?

```
-- Un índice secundario sobre salary permite:  
SELECT * FROM instructor  
WHERE salary = 80000;  
-- Aunque el archivo esté ordenado por ID,  
-- el índice secundario localiza por salary.
```



*Un secondary index localiza lo que el archivo no puede encontrar solo
– pero cada registro encontrado tiene su propio precio en disco.*

Comparación: clustering vs. secondary index

- La diferencia entre ambos no es solo conceptual: tiene un impacto directo en el **número de operaciones de disco** requeridas para responder una consulta.

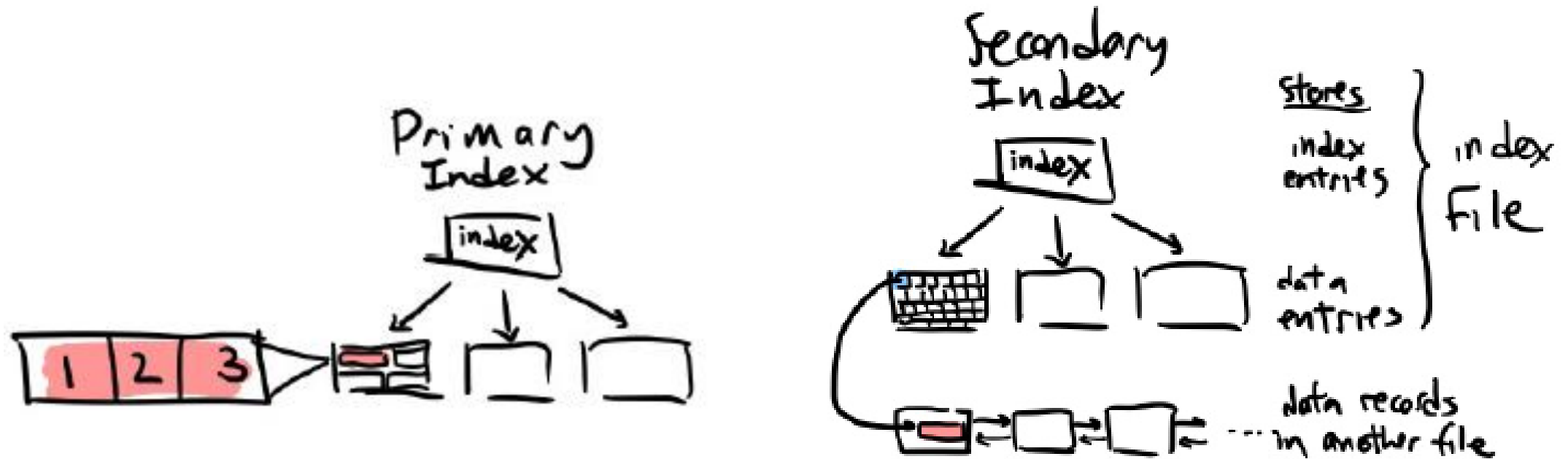
	Clustering index	Secondary index
Orden	Coincide con el archivo físico	Diferente al archivo físico
Densidad	Puede ser disperso	Debe ser denso
Acceso a rangos	Muy eficiente	Costoso
Acceso por igualdad	Eficiente	Eficiente
Cantidad por tabla	Solo uno	Puede haber varios
Operaciones de disco	Mínimas (registros contiguos)	Puede ser una por registro

- Una tabla solo puede tener **un índice de agrupamiento**, ya que los registros solo pueden estar físicamente ordenados de una manera.
- Sin embargo, puede tener **múltiples índices secundarios**, uno por cada atributo de interés.

El clustering index define cómo vive el archivo en disco. Los índices secundarios son ventanas adicionales para acceder a él desde otras perspectivas.

Comparación: clustering vs. secondary index

- La diferencia entre ambos no es solo conceptual: tiene un impacto directo en el **número de operaciones de disco** requeridas para responder una consulta.



Primary

Data entries contain actual tuples

Pros: directly access data.

Secondary

Data entries contain $\langle \text{search key val}, \text{rid} \rangle$ pointers into another file

Pros: index is more compact

Ejemplo 3 - Clustering index

Usando la misma tabla de Estudiantes (1000000 registros, 200000 bloques, 5 registros/bloque, 10 ms/bloque), asuma que el archivo está físicamente ordenado por ID y que existe un índice disperso sobre ID con una entrada por bloque (el índice que ya se analizó en la sección anterior).

Un analista necesita recuperar los estudiantes con ID entre 500001 y 500100.

```
SELECT *  
FROM Estudiantes  
WHERE ID BETWEEN 500001 AND 500100;
```

1. ¿Por qué este índice sobre ID es un clustering index? ¿Qué propiedad del archivo lo hace posible?
2. ¿Cuántos registros contiene el rango buscado? ¿En cuántos bloques de datos consecutivos están almacenados?
3. ¿Cuál es el costo total en I/Os y en tiempo para responder esta consulta usando el índice disperso?
4. ¿Cuál sería el costo si se hiciera un full scan sin índice? ¿Qué concluye al comparar ambos resultados?

Ejemplo 3 – Clustering index

Datos del problema:

- $N = 1000000$ registros
- 5 registros/bloque
- $B = 200000$ bloques
- 10 ms/bloque
- Archivo ordenado físicamente por ID
- 4 carreras con igual distribución
- Cada bloque del índice almacena 100 entradas
- Índice disperso sobre ID: 200,000 entradas $\rightarrow$ 2,000 bloques de índice

Ejemplo 3 – Clustering index

1. Es un clustering index porque los registros en el archivo de datos están físicamente ordenados por el mismo campo que define el índice (en este caso, el **ID**).
 - **Propiedad clave:** El orden físico de las filas en el disco coincide con el orden de las entradas en el índice.
 - **Resultado:** Esto permite que el índice sea disperso (una entrada por bloque en lugar de una por registro), ya que si sabemos dónde empieza el Bloque A y el Bloque B, sabemos que todos los **IDs** intermedios están necesariamente en el Bloque A.

2. Bloques ocupados:

$$N_{rango} = 500100 - 500001 + 1 = 100 \text{ registros}$$

$$B_{rango} = 100 \text{ reg} \times \frac{1 \text{ bloque}}{5 \text{ reg}} = 20 \text{ bloques}$$

Ejemplo 3 – Clustering index

3. **Costo I/O de la consulta:** Recordemos que para una consulta de rango con índice disperso, el proceso es:

- **Buscar en el índice:** Encontrar el bloque donde comienza el ID **500001**.
- **Acceder a los datos:** Leer secuencialmente los bloques necesarios.

Es importante tener en cuenta que el número de registros del índice, es igual al número de bloques, es decir:
 $N_{index} = 1000000 \times \frac{1}{5 \text{ registros}} = 200000$ entradas. Si cada bloque del índice almacena 100 entradas (ejemplo anterior), entonces el número de bloques que tiene el índice es $B = 200000 \text{ reg} \times \frac{1 \text{ bloque}}{100 \text{ reg}} = 2000 \text{ bloques}$. Como la consulta es por rango y el índice es clustering, el DBMS:

- Usa el índice para encontrar el primer bloque relevante.
- Luego lee secuencialmente los 20 bloques consecutivos que contienen el rango.

$$I/O_{Total} = I/O_{search} + I/O_{datos}$$

$$I/O_{search} = \lceil \log_2 B \rceil = \lceil \log_2(2000) \rceil = \lceil 10.96 \rceil = 11$$

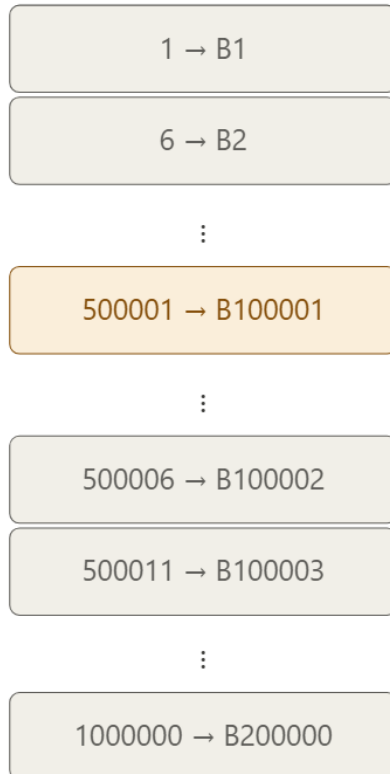
$$I/O_{datos} = 20 \text{ bloques}$$

$$I/O_{Total} = 11 + 20 = 31 \rightarrow T_{total} = 31 \times 10 \text{ ms} = 310 \text{ ms}$$



Ejemplo 3 – Clustering index

Índice disperso (ID)
2,000 bloques — 1 entrada/bloque



Búsqueda binaria
 $\lceil \log_2 2000 \rceil = 11$ I/Os

Archivo de datos
200,000 bloques — ordenado por ID



20
bloques

I/O total = 11 (búsqueda binaria) + 20 (bloques consecutivos) = 31 I/Os → 310 ms
vs. full scan: 200,000 I/Os ≈ 33 min — mejora de ×6,452



Ejemplo 3 – Clustering index

3. **Costo I/O de la consulta:** Recordemos que para una consulta de rango con índice disperso, el proceso es:

- **Buscar en el índice:** Encontrar el bloque donde comienza el ID **500001**.
- **Acceder a los datos:** Leer secuencialmente los bloques necesarios.

Es importante tener en cuenta que el número de registros del índice, es igual al número de bloques, es decir:
 $N_{index} = 1000000 \times \frac{1}{5 \text{ registros}} = 200000$ entradas. Si cada bloque del índice almacena 100 entradas (ejemplo anterior), entonces el número de bloques que tiene el índice es $B = 200000 \text{ reg} \times \frac{1 \text{ bloque}}{100 \text{ reg}} = 2000 \text{ bloques}$. Como la consulta es por rango y el índice es clustering, el DBMS:

- Usa el índice para encontrar el primer bloque relevante.
- Luego lee secuencialmente los 20 bloques consecutivos que contienen el rango.

$$I/O_{Total} = I/O_{search} + I/O_{datos}$$

$$I/O_{search} = \lceil \log_2 B \rceil = \lceil \log_2(2000) \rceil = \lceil 10.96 \rceil = 11$$

$$I/O_{datos} = 20 \text{ bloques}$$

$$I/O_{Total} = 11 + 20 = 31 \rightarrow T_{total} = 31 \times 10 \text{ ms} = 310 \text{ ms}$$



Ejemplo 4 - Secondary index

Sobre la misma tabla de Estudiantes, ahora se construye un índice sobre el atributo Carrera. El archivo sigue ordenado físicamente por ID, y se sabe que las cuatro carreras (Sistemas, Matemáticas, Física y Química) tienen la misma cantidad de estudiantes.

1. ¿Por qué este índice sobre Carrera es un secondary index (índice no agrupado)? ¿Qué característica del archivo lo determina?
2. ¿Cuántas entradas tiene este índice? ¿Cuántos bloques ocupa si cada bloque almacena 100 entradas?
3. Un usuario ejecuta la consulta `WHERE Carrera = 'Sistemas'`. ¿Cuántos registros debe recuperar? En el peor caso, ¿cuántos bloques de datos distintos debe leer el DBMS? ¿Por qué?

```
SELECT *  
FROM Estudiantes  
WHERE Carrera = 'Sistemas';
```

4. Compare este costo con el del full scan. ¿Qué conclusión práctica obtiene sobre el uso de secondary indexes en atributos con baja selectividad?

Ejemplo 4 – Secondary index

Datos del problema:

- $N = 1000000$ registros
- 5 registros/bloque
- $B = 200000$ bloques
- 10 ms/bloque
- Archivo ordenado físicamente por ID
- 4 carreras con igual distribución
- Cada bloque del índice almacena 100 entradas

Ejemplo 4 – Secondary index

1. El índice sobre Carrera es un secondary index (índice no agrupado) por que el archivo está físicamente ordenado por ID, no por Carrera de modo que el orden de las entradas del índice (por Carrera) difiere del orden físico del archivo. Como consecuencia, los registros de una misma carrera no están físicamente contiguos en disco: están dispersos a lo largo de los 200,000 bloques.
2. **Entradas y bloques del índice:**

$$N_{index} = N = 1000000 \text{ reg}$$

$$B_I = \frac{N_{index}}{N_{B_I}} = 1000000 \text{ reg} \times \frac{1 \text{ bloque}}{100 \text{ reg}} = 10000 \text{ bloques}$$

Ejemplo 4 – Secondary index

3. Costo I/O de la consulta WHERE Carrera = 'Sistemas':

Como hay 4 carreras con igual distribución el número de registros por carrera, y más específicamente para sistemas (carrera de interés), está dado por:

$$N_{carrera_i} = \frac{1000000}{4} = 250000 = N_{sistemas}$$

Como el archivo está ordenado por ID y no por Carrera, los 250000 registros de Sistemas están dispersos por todo el archivo. En el peor caso, cada registro está en un bloque distinto:

$$I/O_{Total} = I/O_{search} + I/O_{datos}$$

$$I/O_{search} = \lceil \log_2 B \rceil = \lceil \log_2(10000) \rceil = \lceil 13.28 \rceil = 14$$

$$I/O_{datos} = 250000 \text{ bloques (peor caso)}$$

$$I/O_{Total} = 14 + 250000 = 250014 \rightarrow T_{total} = 250014 \times 10 \text{ ms} = 2500140 \text{ ms} \approx 41.7 \text{ min}$$



Ejemplo 4 – Secondary index

4. Comparación con full scan:

$$T_{FS} = 1000000 \text{ reg} \times \frac{1 \text{ bloque}}{5 \text{ reg}} \times \frac{10 \text{ ms}}{1 \text{ bloque}} \times \frac{1 \text{ s}}{1000 \text{ ms}} \times \frac{1 \text{ min}}{60} \approx 33 \text{ min}$$

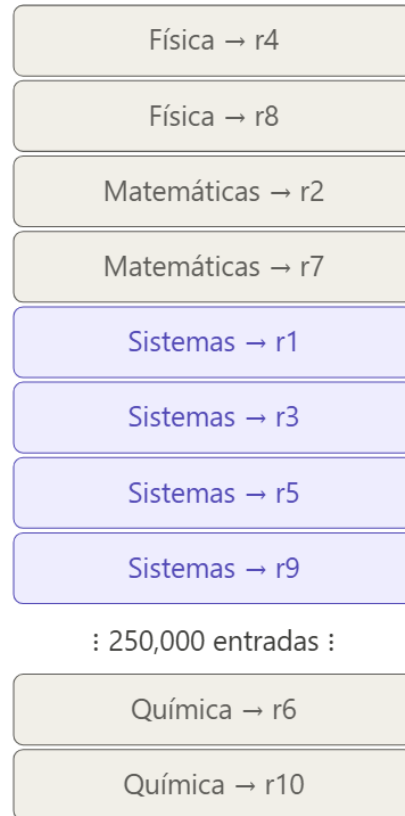
	I/Os	Tiempo
Secondary index	250,014	~41.7 min
Full scan	200,000	~33 min

Conclusión: El secondary index sobre Carrera es en este caso más lento que hacer un full scan. Esto ocurre porque Carrera tiene baja selectividad — el 25% de los registros coincide con la consulta. Con tanta coincidencia y registros dispersos físicamente, el DBMS termina haciendo más I/Os que si simplemente leyera todos los bloques del archivo en orden. Este es el trade-off central del **secondary index**: es muy eficiente cuando se busca un valor raro (alta selectividad), pero puede ser contraproducente cuando el resultado es grande.

Ejemplo 4 – Secondary index

Índice denso (Carrera)

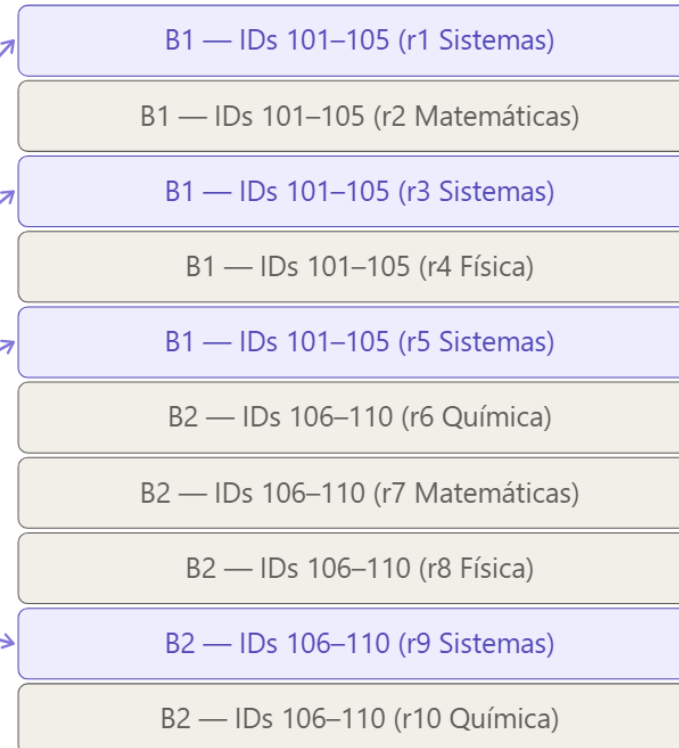
1 entrada / registro



Búsqueda binaria
 $\lceil \log_2 10000 \rceil = 14$ I/Os

Archivo de datos

ordenado por ID (no por Carrera)



: 199,998 bloques más :

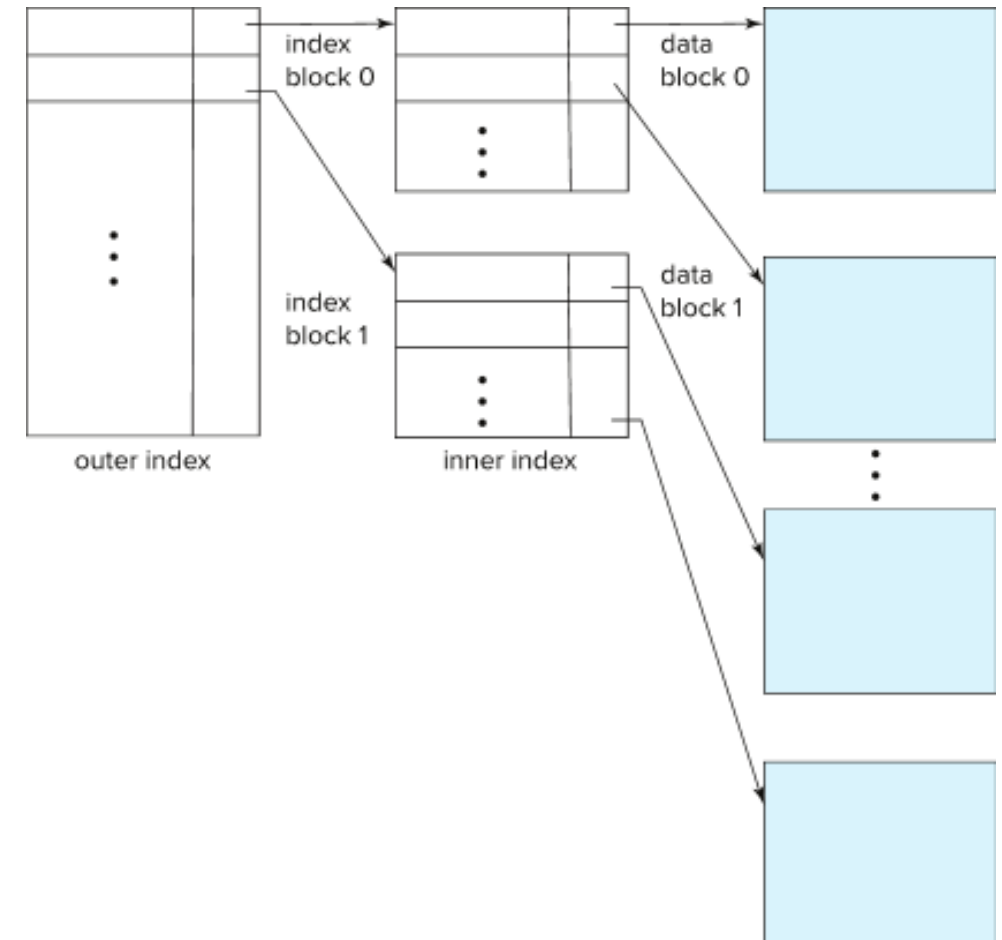
I/O total = 14 + 250,000 = 250,014 I/Os ≈ 41.7 min

vs. full scan: 200,000 I/Os ≈ 33 min — el secondary index es más lento



Cuando el índice también necesita un índice

- Hasta ahora hemos asumido que el índice cabe completamente en memoria. Pero ¿qué ocurre cuando el índice es tan grande que **tampoco cabe en memoria**?
- La solución es aplicar el mismo principio de indexación sobre el propio índice: construir un índice **sobre el índice**.
- A esta estructura se le denomina **índice multinivel** (*multilevel index*) y se organiza en dos capas:
 - **Índice interno (inner index)**: el archivo de índice básico, almacenado en disco.
 - **Índice externo (outer index)**: un índice disperso construido sobre el índice interno.



Cuando el índice también necesita un índice

- Si el índice externo tampoco cabe en memoria, se puede agregar otro nivel más, y así sucesivamente.
- Los índices en todos los niveles deben actualizarse ante cualquier inserción o eliminación en el archivo.

El índice multinivel es la respuesta natural a la escalabilidad: cuando los datos crecen, la estructura de acceso crece con ellos de forma jerárquica.





Ejemplo 5 – Índice multinivel

Siguiendo el contexto del ejemplo anterior, el índice denso sobre ID tiene 10,000 bloques. Suponga que ese índice es demasiado grande para caber en memoria principal. Se decide construir un **índice multinivel**: un índice externo (**outer index**) disperso sobre el índice interno (**inner index**) denso, con una entrada por cada bloque del índice interno.

1. ¿Cuántas entradas tiene el outer index? ¿Cuántos bloques ocupa si cada bloque almacena 100 entradas?
2. ¿Por qué el outer index sí cabe en memoria y el inner index no necesariamente?
3. ¿Cuál es el costo en I/Os para recuperar el registro con ID = 500,001 usando el índice multinivel?
4. Compare el costo con el índice denso de un solo nivel, el disperso de un solo nivel y el full scan. ¿Qué concluye?





Ejemplo 5 – Índice multinivel

Datos del problema:

- $N = 1000000$ registros
- 5 registros/bloque
- $B = 200000$ bloques
- 10 ms/bloque
- Archivo ordenado físicamente por ID
- 4 carreras con igual distribución
- Cada bloque almacena 100 entradas | 10 ms/bloque
- Índice disperso sobre ID: 200,000 entradas $\rightarrow$ 2,000 bloques de índice
- **Índice denso sobre ID (inner index):** 1,000,000 entradas $\rightarrow$ 10,000 bloques
- **Outer index:** disperso, 1 entrada por bloque del inner index



Ejemplo 5 – Índice multinivel

1. Entradas y bloques del outer index

$$N_{outer} = B_{inner} = 10000 \text{ entradas}$$

$$B_{outer} = \frac{N_{outer}}{N_{B_I}} = \frac{10000}{100} = 100 \text{ bloques}$$

2. ¿Por qué el outer cabe en memoria?

El inner index ocupa 10000 bloques — potencialmente varios MB dependiendo del tamaño de bloque, demasiado para garantizar su presencia en memoria. El outer index en cambio ocupa solo 100 bloques, un tamaño suficientemente pequeño para mantenerse en memoria principal de forma permanente. Por eso la búsqueda sobre el outer index no genera ninguna operación de disco.

Ejemplo 5 – Índice multinivel

3. Costo I/O para recuperar ID = 500001

El proceso tiene tres pasos:

Paso 1 – Buscar en el outer index (en memoria, sin I/Os de disco):

$$I/O_{outer} = 0 \text{ (Esta en memoria)}$$

Paso 2 – Leer el bloque del inner index apuntado por el outer:

$$I/O_{inner} = 1$$

Paso 3 – Leer el bloque de datos apuntado por el inner index:

$$I/O_{datos} = 1$$

$$I/O_{Total} = I/O_{outer} + I/O_{inner} + I/O_{datos} = 0 + 1 + 1 = 2$$

$$T_{Total} = 2 \times 10 \text{ ms} = 20 \text{ ms}$$



Ejemplo 5 – Índice multinivel

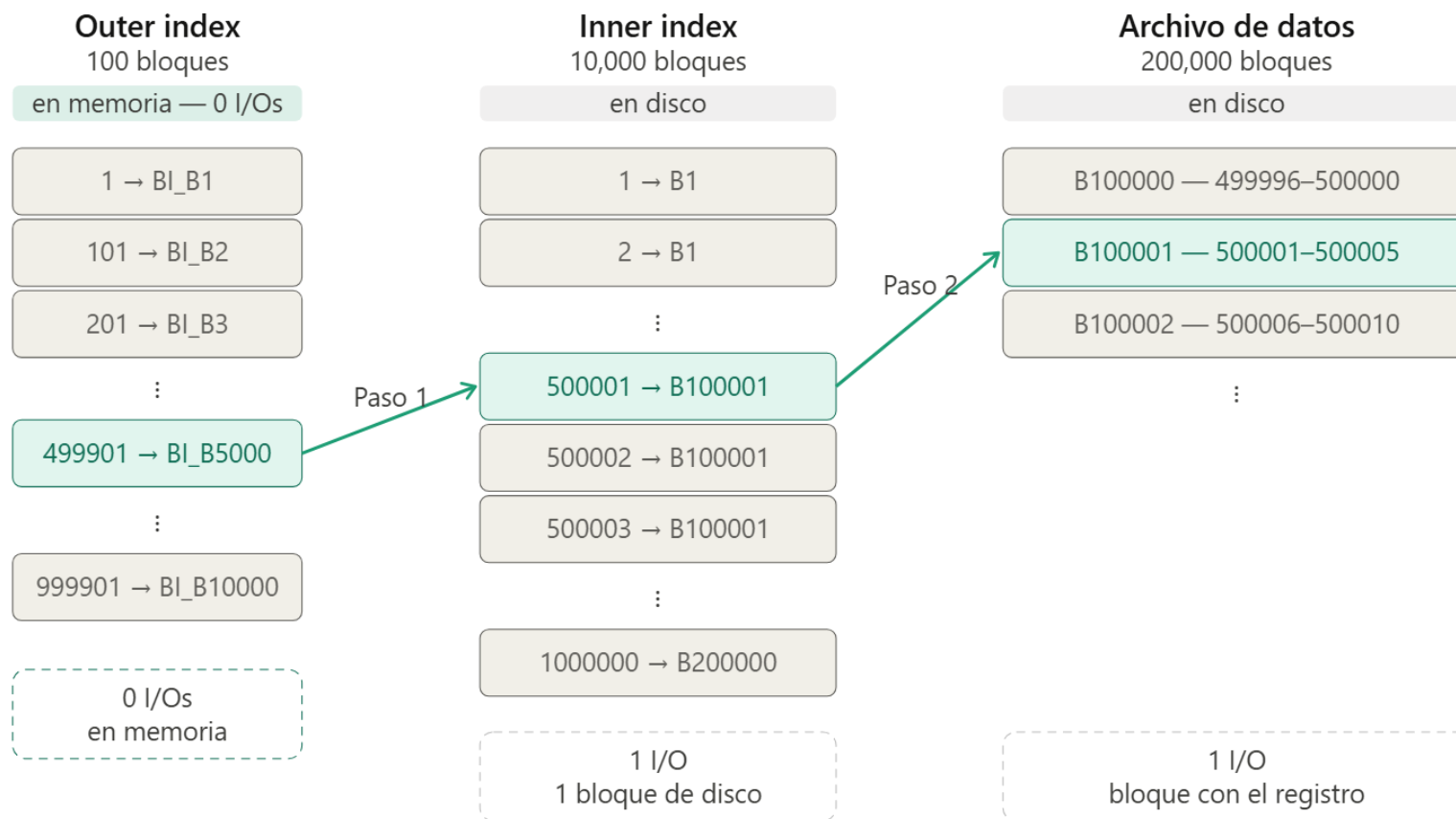
4. Tabla comparativa final

	Bloques del índice	I/Os	Tiempo
Índice multinivel	100 (outer) + 10000 (inner)	2	20 ms
Índice denso (1 nivel)	10,000	15	150 ms
Índice disperso (1 nivel)	2,000	12	120 ms
Sin índice	—	200000	~33 min

Conclusión: El índice multinivel logra el menor costo de búsqueda de todos los casos — solo 2 I/Os — porque elimina la búsqueda binaria sobre el inner index al mantener el outer index en memoria. El trade-off es que ocupa más espacio total (10,100 bloques de índice vs 2,000 del disperso), pero la ganancia en velocidad es significativa. Esta lógica de construir índices sobre índices es exactamente el principio detrás de los árboles B⁺, que generalizan esta idea de forma dinámica y balanceada.



Ejemplo 5 – Índice multinivel



I/O total = 0 + 1 + 1 = 2 I/Os → 20 ms

Outer en memoria (0 I/Os) → 1 bloque inner → 1 bloque de datos

Camino de búsqueda para ID = 500,001

Entradas / bloques no accedidos



Actualización de índices

- Sin importar el tipo de índice que se esté usando, los índices se deben actualizar siempre que se inserte o borre un registro del archivo.
- Si un registro del archivo es modificado, cualquier índice cuyo atributo de clave de búsqueda se vea afectado por dicha modificación también debe actualizarse:
 - Si se cambia el departamento de un profesor, un índice sobre el atributo **dept_name** de la relación **instructor** debe actualizarse de forma correspondiente.
- Una actualización se modela como:
 1. **Eliminación** del registro antiguo
 2. **Inserción** del nuevo valor del registro
 3. **Eliminación** del índice seguida de una **inserción** en el índice.
- Debido a lo anterior, solo es necesario considerar inserción y eliminación sobre un índice; las actualizaciones no se tratan de forma explícita.

10101		10101	Srinivasan	Comp. Sci.	65000	
12121		12121	Wu	Finance	90000	
15151		15151	Mozart	Music	40000	
22222		22222	Einstein	Physics	95000	
32343		32343	El Said	History	60000	
33456		33456	Gold	Physics	87000	
45565		45565	Katz	Comp. Sci.	75000	
58583		58583	Califieri	History	62000	
76543		76543	Singh	Finance	80000	
76766		76766	Crick	Biology	72000	
83821		83821	Brandt	Comp. Sci.	92000	
98345		98345	Kim	Elec. Eng.	80000	

10101		10101	Srinivasan	Comp. Sci.	65000	
32343		12121	Wu	Finance	90000	
76766		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		32343	El Said	History	60000	
		33456	Gold	Physics	87000	
		45565	Katz	Comp. Sci.	75000	
		58583	Califieri	History	62000	
		76543	Singh	Finance	80000	
		76766	Crick	Biology	72000	
		83821	Brandt	Comp. Sci.	92000	
		98345	Kim	Elec. Eng.	80000	

Actualización de índices - Eliminación

- Para eliminar un registro, el sistema primero localiza el registro a eliminar.
- Las acciones siguientes dependen de si el índice es denso o disperso.

Indices densos

1. Si el registro eliminado era el **único** con ese valor de clave de búsqueda, el sistema elimina la entrada correspondiente del índice.
2. De lo contrario, se toman las siguientes acciones:
 - a. Si la entrada del índice **almacena punteros a todos los registros** con el mismo valor de clave de búsqueda, el sistema elimina el puntero al registro borrado.
 - b. Si la entrada del índice **almacena únicamente un puntero al primer** registro con ese valor de clave de búsqueda, y el registro eliminado era ese primer registro, el sistema actualiza la entrada para apuntar al siguiente registro.

10101	◀	10101	Srinivasan	Comp. Sci.	65000
12121	◀	12121	Wu	Finance	90000
15151	◀	15151	Mozart	Music	40000
22222	◀	22222	Einstein	Physics	95000
32343	◀	32343	El Said	History	60000
33456	◀	33456	Gold	Physics	87000

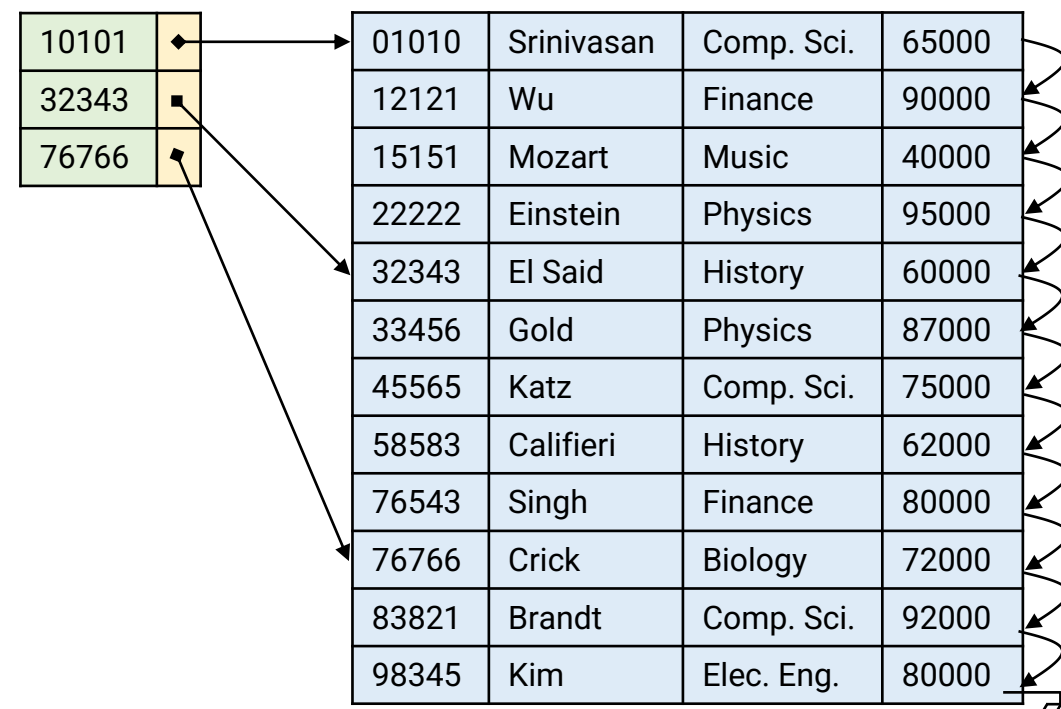
Comp. Sci.	◀	10101	Srinivasan	Comp. Sci.	65000
Finance	◀	12121	Wu	Finance	90000
Music	◀	15151	Mozart	Music	40000
Physics	◀	22222	Einstein	Physics	95000
History	◀	32343	El Said	History	60000
Physics	◀	33456	Gold	Physics	87000

Actualización de índices - Eliminación

- Para eliminar un registro, el sistema primero localiza el registro a eliminar.
- Las acciones siguientes dependen de si el índice es denso o disperso.

Índices dispersos

1. Si el índice **no contiene** una entrada con el valor de clave de búsqueda del registro eliminado, no es necesario realizar ningún cambio en el índice.
2. De lo contrario, el sistema toma las siguientes acciones:
 - a. Si el registro eliminado era el **único** con esa clave de búsqueda, el sistema **reemplaza** la entrada del índice por una entrada con el siguiente valor de clave de búsqueda (en orden de clave de búsqueda). Si el siguiente valor ya tiene una entrada en el índice, la entrada se **elimina** en lugar de ser reemplazada.
 - b. Si la entrada del índice apunta al registro que se está eliminando, el sistema **actualiza** dicha entrada para apuntar al siguiente registro con el mismo valor de clave de búsqueda.



Actualización de índices - Inserción

- Para los índices de un solo nivel, se realiza una búsqueda usando el valor de clave de búsqueda del registro a insertar.

10101		10101	Srinivasan	Comp. Sci.	65000	
12121		12121	Wu	Finance	90000	
15151		15151	Mozart	Music	40000	
22222		22222	Einstein	Physics	95000	
32343		32343	El Said	History	60000	
33456		33456	Gold	Physics	87000	
45565		45565	Katz	Comp. Sci.	75000	
58583		58583	Califieri	History	62000	
76543		76543	Singh	Finance	80000	
76766		76766	Crick	Biology	72000	
83821		83821	Brandt	Comp. Sci.	92000	
98345		98345	Kim	Elec. Eng.	80000	

10101		10101	Srinivasan	Comp. Sci.	65000	
32343		12121	Wu	Finance	90000	
76766		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		32343	El Said	History	60000	
		33456	Gold	Physics	87000	
		45565	Katz	Comp. Sci.	75000	
		58583	Califieri	History	62000	
		76543	Singh	Finance	80000	
		76766	Crick	Biology	72000	
		83821	Brandt	Comp. Sci.	92000	
		98345	Kim	Elec. Eng.	80000	

Actualización de índices - Inserción

- Primero, el sistema realiza una búsqueda usando el valor de clave de búsqueda del registro a insertar.
- Las acciones siguientes dependen de si el índice es denso o disperso.

Indices densos

- Si el valor de clave **de búsqueda** no aparece en el índice, el sistema inserta una nueva entrada en la posición correspondiente.
- De lo contrario, se toman las siguientes acciones:
 - Si la entrada almacena **punteros a todos los registros** con el mismo valor, el sistema agrega un puntero al nuevo registro en la entrada existente.
 - Si la entrada almacena **únicamente un puntero al primer registro**, el sistema simplemente ubica el nuevo registro después de los demás registros con el mismo valor de clave de búsqueda.

10101	◀	10101	Srinivasan	Comp. Sci.	65000
12121	◀	12121	Wu	Finance	90000
15151	◀	15151	Mozart	Music	40000
22222	◀	22222	Einstein	Physics	95000
32343	◀	32343	El Said	History	60000
33456	◀	33456	Gold	Physics	87000

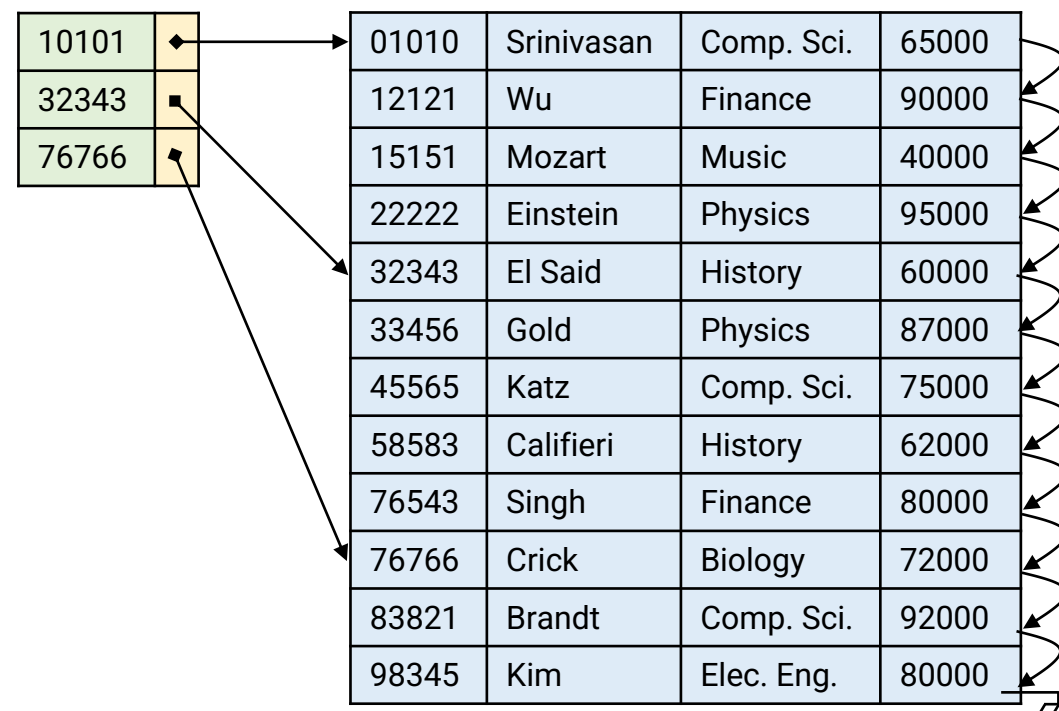
Comp. Sci.	◀	10101	Srinivasan	Comp. Sci.	65000
Finance	◀	12121	Wu	Finance	90000
Music	◀	15151	Mozart	Music	40000
Physics	◀	22222	Einstein	Physics	95000
History	◀	32343	El Said	History	60000
Physics	◀	33456	Gold	Physics	87000

Actualización de índices - Inserción

- Primero, el sistema realiza una búsqueda usando el valor de clave de búsqueda del registro a insertar.
- Las acciones siguientes dependen de si el índice es denso o disperso.

Indices dispersos

- Si se crea un **nuevo bloque**, el primer valor de clave de búsqueda del nuevo bloque se inserta en el índice.
- Si el nuevo registro tiene el **menor valor de clave en su bloque**, se actualiza la entrada del índice para apuntar al bloque.
- Si **no** tiene el menor valor, no se realiza ningún cambio en el índice.



Indices ordenados – Lo que aprendimos

- Los índices ordenados se pueden caracterizar a lo largo de **tres dimensiones** que determinan su comportamiento, costo y eficiencia:

Dimensión	Tipos	Criterio de elección
Densidad	Denso / Disperso	Velocidad vs. espacio
Agrupamiento	Clustering / Secondary	Orden físico del archivo
Niveles	Un nivel / Multinivel	Tamaño del índice vs. memoria

- Estas tres dimensiones **no son independientes**: la elección en una afecta las opciones en las otras.
 - Un índice secundario **debe ser denso**.
 - Un índice disperso **requiere** que el archivo esté ordenado.
 - Un índice multinivel surge cuando el índice de **un solo nivel** no cabe en memoria.
- En la práctica, la mayoría de los DBMS implementan los índices ordenados mediante **árboles B⁺ (B⁺-Tree)**, que incorporan estas tres dimensiones de forma integrada y eficiente.

Las tres dimensiones vistas no son categorías aisladas del libro – son las decisiones de diseño reales que todo motor de base de datos debe resolver.

Referencias

- <https://www.mongodb.com/resources/basics/databases/nosql-explained>
- <https://blog.bytebytego.com/p/sql-vs-nosql-choosing-the-right-database>
- <https://cs186berkeley.net/notes/note17/>
- <https://levelup.gitconnected.com/postgresql-deep-dive-the-anatomy-of-disk-file-and-page-layout-part-2-9b6fb7ede383>
- <https://medium.com/@pablo.lopez.santori/what-do-postgres-tables-actually-look-like-426028eabc1a>
- <https://michalpittr.substack.com/p/how-does-sqlite-store-data>
- <https://cs186berkeley.net/notes/note3/>
- <https://github.com/MIT-DB-Class/simple-db-hw>
- <https://simplifiedb-java.netlify.app/#/>
- <https://github.com/quazi-irfan/pySimpleDB>
- <https://tinydb.readthedocs.io/en/latest/>

UNIVERSIDAD DE ANTIOQUIA

Curso de Estructuras de datos
Clase 7 – Indexación